

第三讲 最易解释的机器学习预测算法

---决策树算法

主要内容

- 继续讨论**案例二**的**回归预测**问题
 - 从K-近邻法到决策树
 - 讨论基于决策树的回归预测
 - 决策树的基本概念，非线性特征
 - 讨论决策树的几个核心问题
- 讨论**案例五**的**多分类预测**问题
 - 讨论基于决策树的分类预测和非线性特征
 - 案例的思考
- 决策树算法的**思考**

案例二：共享单车日租赁量的预测

- **案例二：**共享单车日租赁量(Casual/Registered/count)的预测
- **案例二数据：**

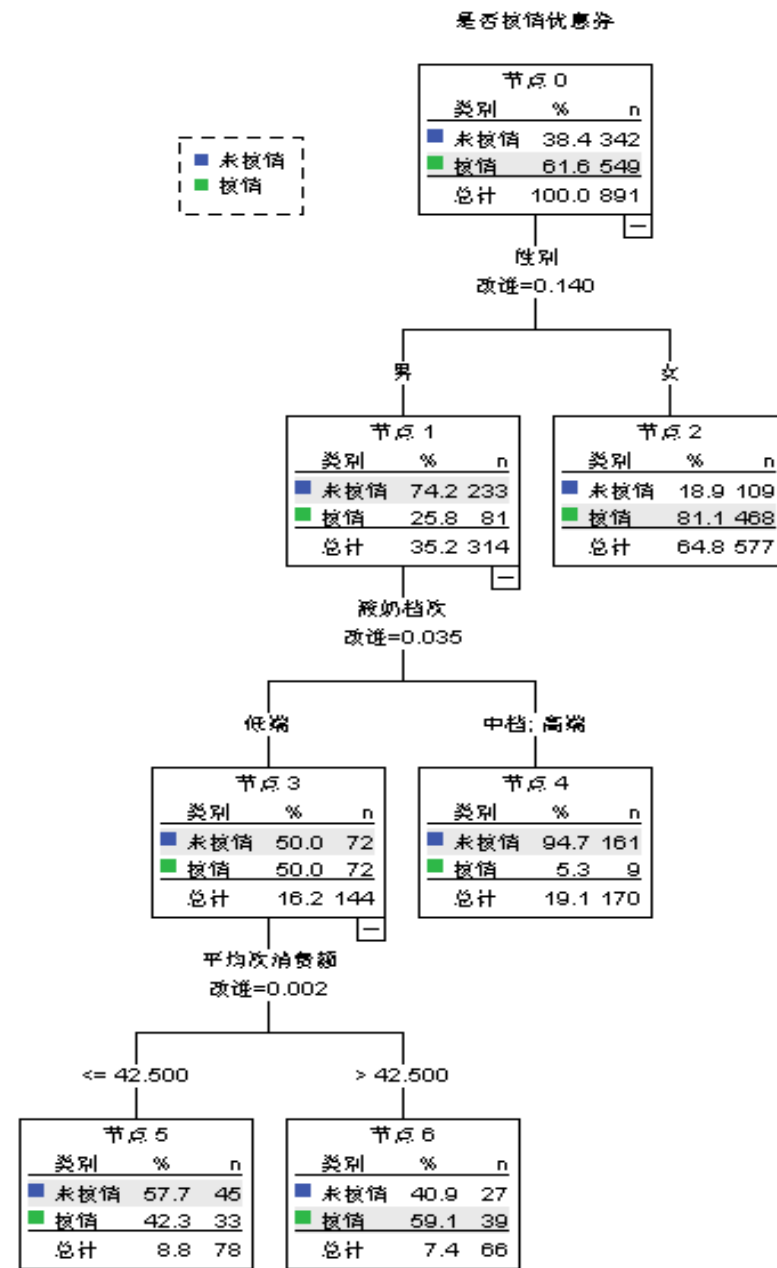
	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- Season – 1:spring 2:summer 3:fall 4:winter
- Holiday – whether the day is considered a holiday
- Workingday: whether the day is neither a weekend nor holiday
- Weather -
 - 1: Clear, Few clouds, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- Temp - temperature in Celsius
- Atemp - "feels like" temperature in Celsius
- Humidity - relative humidity
- Windspeed - wind speed
- Casual – number of non-registered user rentals initiated
- Registered - number of registered user rentals initiated
- count - number of total rentals

- 案例二的分析：一般线性回归模型→K-近邻法
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)
 - 问两大问题：
 - 问题一：如何合理引用分类型输入变量？
 - 问题二：如何确定重要影响因素？
 - 问题一：如何合理引用分类型输入变量？解决思路：
 - 分类型和数值型变量做统一处理
 - 将数值型输入变量离散化为分类型变量
 - 沿用K-近邻法的预测思路
 - 但将“一组近邻”视为多个分类型变量交叉分组下的个“一个分组”
 - 回归预测结果为“分组”内输出变量的平均值
 - 分类预测结果为“分组”内输出变量的众数类
 - →决策树算法

决策树的基本概念

- 决策树：得名于其分析结果的展示方式类似一棵倒置的树，分为：
 - 分类树：分类预测；回归树：回归预测
- 概念：
 - 树深度：是树根到树叶的最大层数，通常作为决策树模型复杂度的一种度量
 - 树节点和树分枝：
 - 每个父节点下均仅有两个子节点的决策树称为**2叉树**，否则为**多叉树**
 - 根据节点所在层，节点由上至下分为**根节点、中间节点和叶节点**



- 决策树是数据**反复分组**的图形化体现
- 决策树是推理规则的图形化展示
 - 决策树中每个节点都应一条**推理规则**
 - 推理规则通过逻辑判断的形式反映输入变量和输出变量之间的取值规律**
- 决策树的预测
 - 基于叶节点的推理规则**实现对新数据的分类预测或回归预测

- 对于样本观测 X_0 ，预测只需从根节点开始依次根据输入变量取值，沿着决策树的不同分枝“走到”样本量等于 n_k 的叶节点 $Lnode_k$

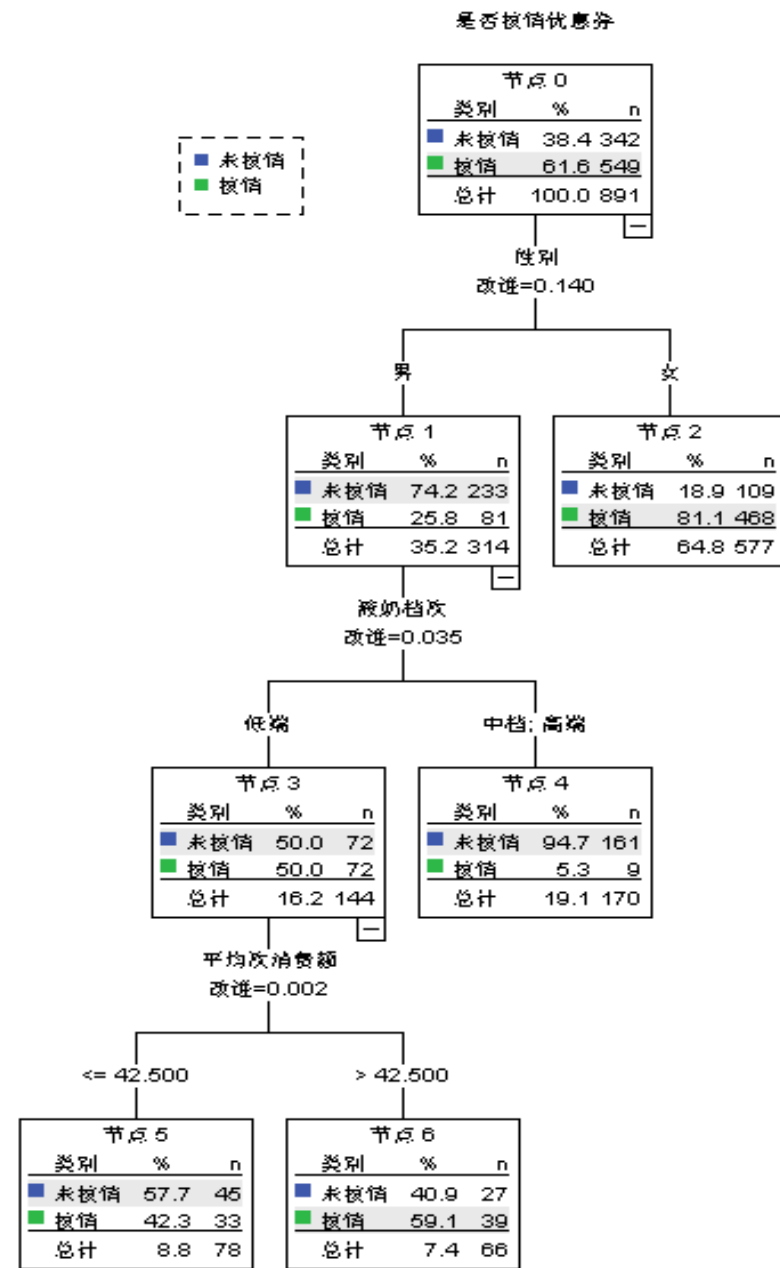
- 对于分类树：

$$\hat{y}_0 = \text{mode}(y_i), i: X_i \in Lnode_k$$

- 规则**置信度**：众数类占比 $\frac{n_{y_k}}{n_k}$

- 对于回归树：

$$\hat{y}_0 = \frac{1}{n_k} \sum_{i: X_i \in Lnode_k} y_i$$



- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)

```
X = data[['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed']]
for Y in ['casual', 'registered']:
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    modelDTC = tree.DecisionTreeRegressor(random_state=123)
    modelDTC.fit(X_train, y_train)
    print('\n对%s的预测(训练MAE=%f;测试MAE=%f)' % (Y, mean_absolute_error(y_train, modelDTC.predict(X_train)),
                                                    mean_absolute_error(y_test, modelDTC.predict(X_test))))

plt.figure(figsize = (15, 10))
tree.plot_tree(modelDTC, feature_names = list(X), class_names = np.unique(y), filled=True)
```

决策树

对casual的预测(训练MAE=2.569622;测试MAE=22.440401)

对registered的预测(训练MAE=18.142090;测试MAE=116.650192)

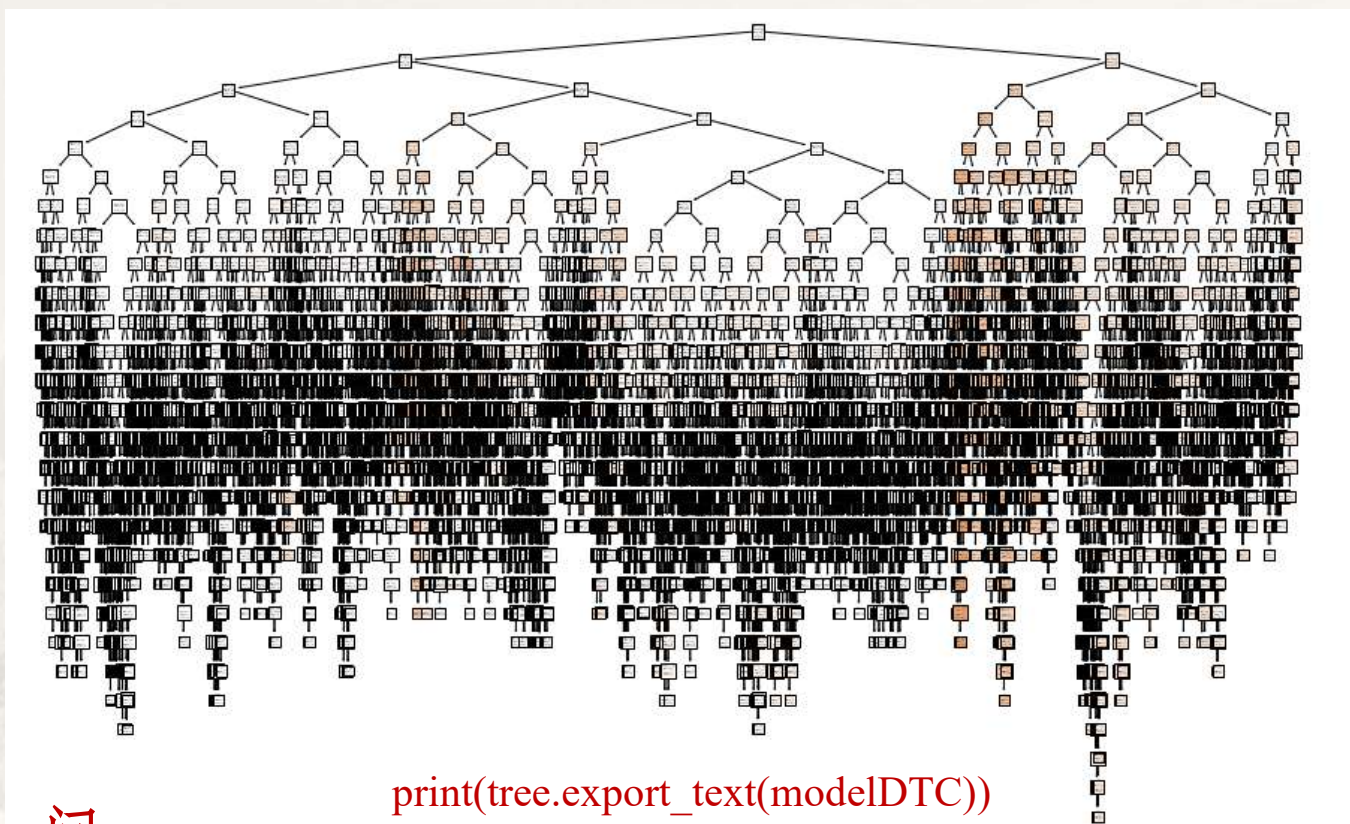
K-近邻(训练误差)

对casual的预测: R方=0.825364;MAE=9.162251

对registered的预测: R方=0.731612;MAE=41.564949

- 问:
 - 为什么训练误差下降较为显著?
 - 从树的特点出发考虑

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)

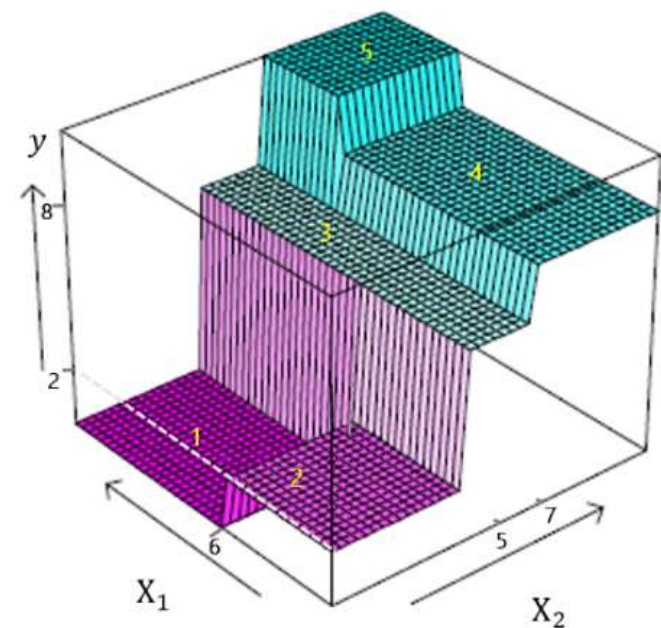
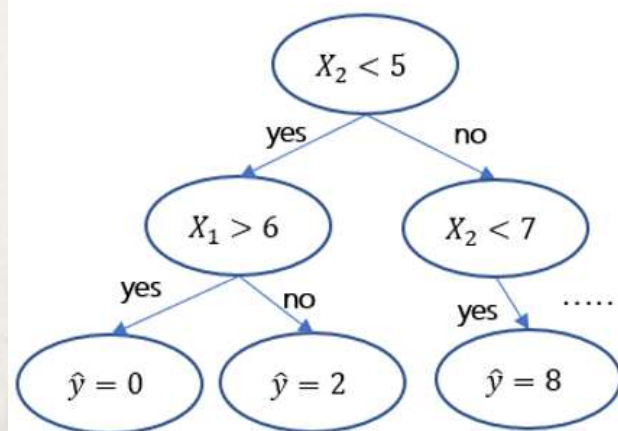


- 问：
 - 为什么训练误差下降较为显著？
 - 树深度大，复杂度高(如何理解?)
 - 决策树的非线性特点

```
— feature_5 <= 29.93
  — feature_4 <= 12.71
    — feature_0 <= 3.00
      — feature_2 <= 0.50
        — feature_6 <= 41.50
          — feature_6 <= 35.50
            — feature_7 <= 29.00
              — feature_1 <= 0.50
                — feature_6 <= 34.50
                  — feature_6 <= 32.50
                    — feature_4 <= 10.66
                      — truncated branch
                    — feature_4 > 10.66
                      — value: [84.00]
                  — feature_6 > 32.50
                    — feature_4 <= 10.25
                      — value: [184.00]
                    — feature_4 > 10.25
                      — value: [229.50]
                — feature_6 > 34.50
                  — feature_7 <= 26.00
                    — feature_5 <= 10.61
                      — value: [77.00]
                    — feature_5 > 10.61
                      — value: [95.00]
                  — feature_7 > 26.00
                    — value: [176.00]
              — feature_1 > 0.50
                — feature_4 <= 10.25
                  — feature_6 <= 33.50
                    — value: [16.00]
                  — feature_6 > 33.50
                    — value: [51.00]
                — feature_4 > 10.25
                  — feature_5 <= 11.74
                    — feature_7 <= 27.00
                      — value: [136.00]
                    — feature_7 > 27.00
                      — value: [144.00]
                  — feature_5 > 11.74
                    — feature_6 <= 34.00
                      — value: [113.00]
                    — feature_6 > 34.00
                      — value: [89.00]
            — feature_7 > 29.00
              — feature_6 <= 33.50
                — feature_4 <= 10.25
                  — feature_7 <= 32.00
```


- 回归预测中的决策树：非线性特点
 - Python模拟和启示：认识决策树的回归面

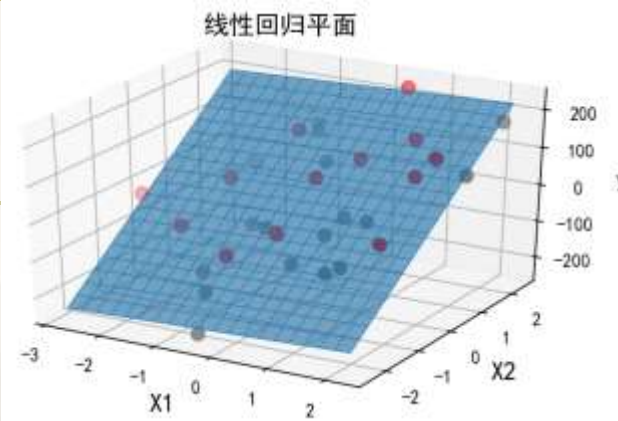
- 数据分组也即：
 - 对 p 个输入变量构成的 p 维实数空间进行反复划分，形成多个不相交的小区域
 - 每个小区域是一个数据分组，对应决策树的一个叶节点



- 问：
 - 图中的各个小平面是如何确定的？
- 小平面的平滑连接将形成一个不规则的曲面，该曲面就是决策树给出的用于回归预测的回归曲面

- 回归预测中的决策树：非线性特点
 - Python模拟和启示：认识决策树的回归面

Chapter6-1.ipynb



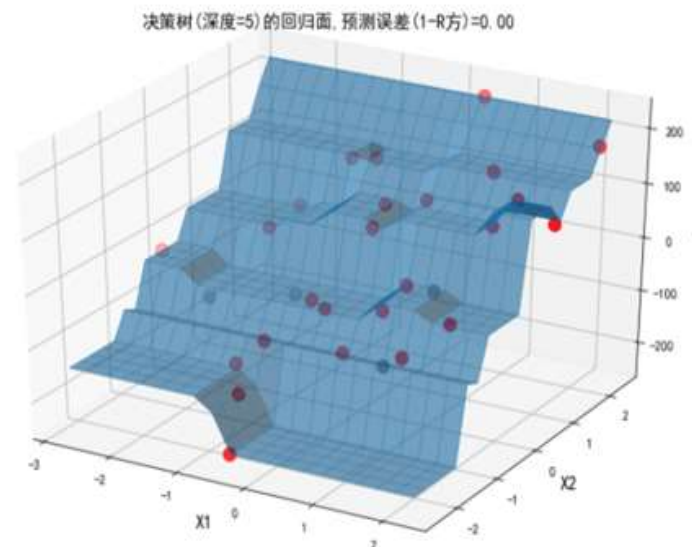
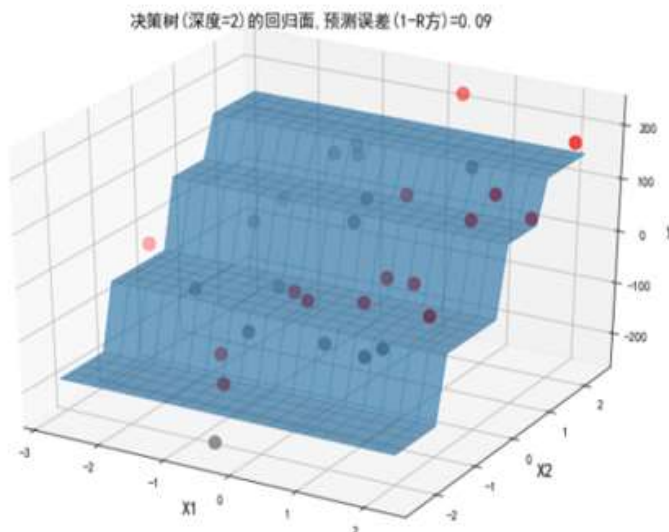
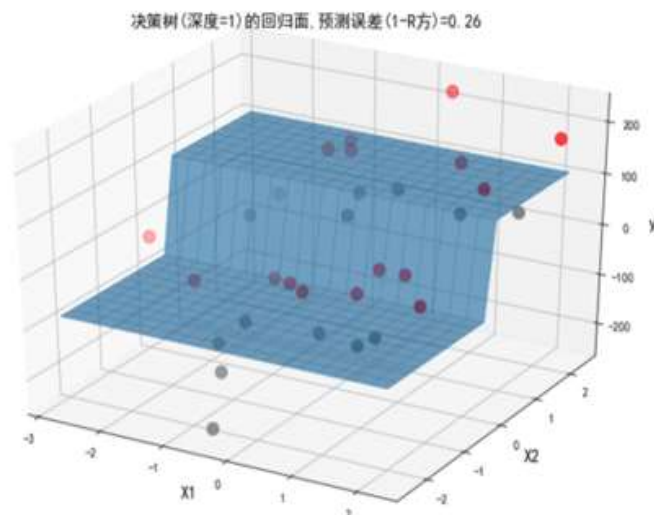
```

N = 30
X, y = make_regression(n_samples=N, n_features=2, n_informative=2, noise=20, random_state=123)
X01, X02 = np.meshgrid(np.linspace(X[:, 0].min(), X[:, 0].max(), 20), np.linspace(X[:, 1].min(), X[:, 1].max(), 20))
X0 = np.hstack((X01.reshape(400, 1), X02.reshape(400, 1)))
def Myplot(y0hat, title, yhat):
    ax = Axes3D(plt.figure(figsize=(9, 6)))
    ax.scatter(X[y>yhat, 0], X[y>yhat, 1], y[y>yhat], c='red', marker='o', s=100)
    ax.scatter(X[y<yhat, 0], X[y<yhat, 1], y[y<yhat], c='grey', marker='o', s=100)
    ax.plot_surface(X01, X02, y0hat.reshape(20, 20), alpha=0.6)
    ax.plot_wireframe(X01, X02, y0hat.reshape(20, 20), linewidth=0.5)
    ax.set_title(title, fontsize=14)
    ax.set_xlabel('X1', fontsize=14)
    ax.set_ylabel('X2', fontsize=14)
    ax.set_zlabel('y', fontsize=14)
for d in [1, 2, 5]:
    modelDTC = tree.DecisionTreeRegressor(max_depth=d, random_state=123)
    modelDTC.fit(X, y)
    Myplot(y0hat=modelDTC.predict(X0), title="决策树(深度=%d)的回归面, 预测误差(1-R方)=%.2f"%(d, 1-modelDTC.score(X, y)), yhat=y)

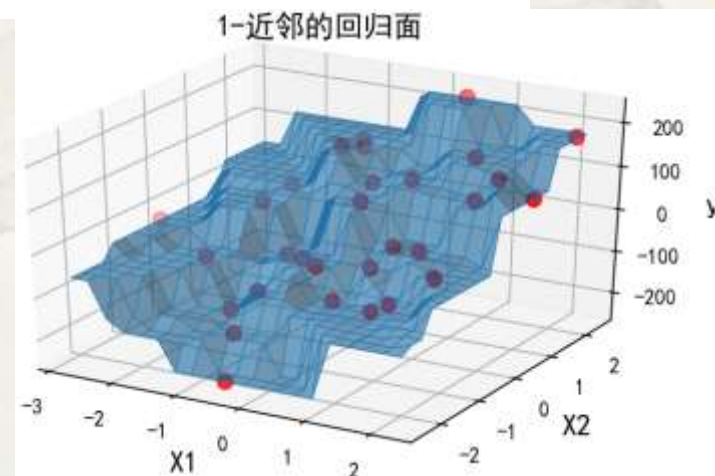
```

后续讲

- 回归预测中的决策树：非线性特点
 - Python模拟和启示：认识决策树的回归面



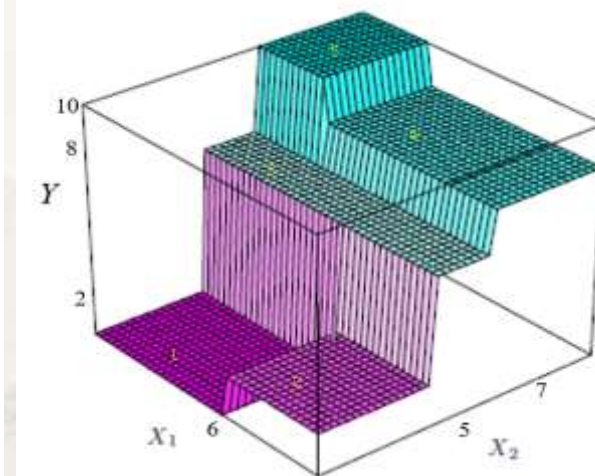
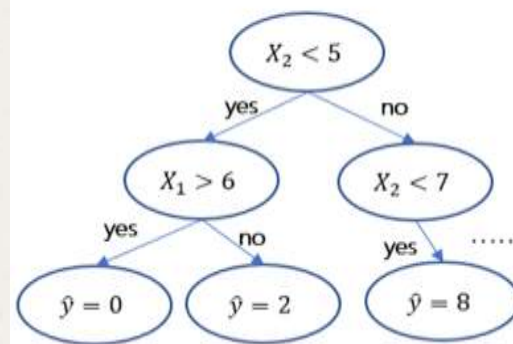
- 问：
 - 决策树的回归面和K-近邻法的回归面的共同特点是什么？
 - 局部和全局
 - 你认为决策树应该解决哪些核心问题？



• 决策树的**核心**问题：依据什么**原则**确定**最佳**分组变量和**组限**？

• 从算法**目标**考虑！回归预测最**关注**什么？

$$\min \sum_{i: X_i \in R_1} (y_i - \hat{y}_{R_1})^2 + \sum_{i: X_i \in R_2} (y_i - \hat{y}_{R_2})^2$$



• 能**最大程度**降低节点内**y**的取值**差异**的变量和组限是**最佳的**

• 空间划分得到的两区域 R_1, R_2 中输出变量的**异质性** (Impurity)——**方差** (回归预测)，将较划分之前下降最大

$$S^2(t) = \frac{1}{N_t} \sum_{i=1}^{N_t} (y_i(t) - \bar{y}(t))^2 \quad \Delta S^2(t) = S^2(t) - \underbrace{\left[\frac{N_{t_{left}}}{N_t} S^2(t_{left}) + \frac{N_{t_{right}}}{N_t} S^2(t_{right}) \right]}_{?}$$

• 决策树的**核心**问题：通过怎样的**途径**实现？

• 从算法**效率**考虑：经典决策树算法CART

• CART(Classification And Regression Tree) 分类回归树

• 采用自顶向下的**递归二分**策略

• 相对上图而言，下图是一种迭代二分策略

• 采用贪心算法确定**当前最佳**分组变量和组限值

• 贪心算法是一种不断寻找当前局部最优解的算法

• **迭代结束即决策树生长结束！**

• 问：

• 基于上述算法，你认为可依据**什么**评价输入变量的重要性？

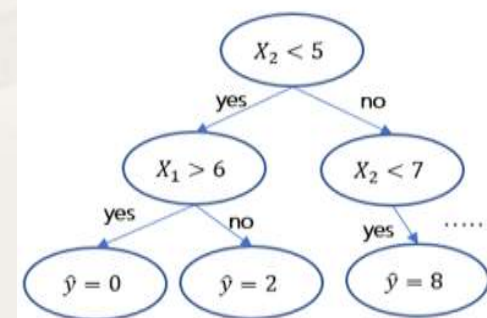
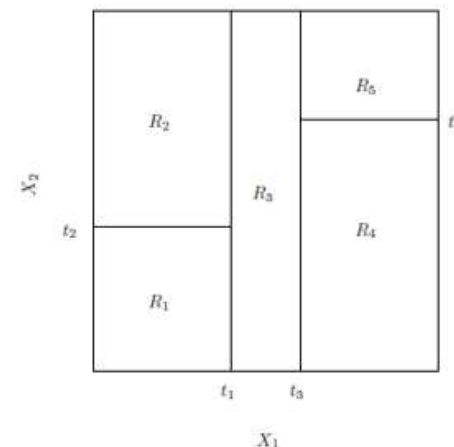
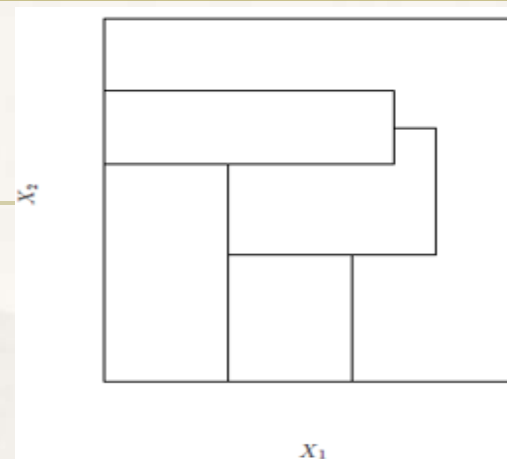
• 越接近**树根**的输入变量**越重要**！

• 决策树的**损失函数**是什么？如何体现y的**监督**作用？

• 决策树的**参数**是什么？

• 决策树的参数**优化策略**是什么？

$$\min \sum_{i: X_i \in R_1} (y_i - \hat{y}_{R_1})^2 + \sum_{i: X_i \in R_2} (y_i - \hat{y}_{R_2})^2$$



- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)

```
X = data[['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed']]
for Y in ['casual', 'registered']:
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    modelDTC = tree.DecisionTreeRegressor(random_state=123)
    modelDTC.fit(X_train, y_train)
    print('\n对%s的预测(训练MAE=%f;测试MAE=%f)' % (Y, mean_absolute_error(y_train, modelDTC.predict(X_train)),
                                                    mean_absolute_error(y_test, modelDTC.predict(X_test))))

plt.figure(figsize = (15, 10))
tree.plot_tree(modelDTC, feature_names = list(X), class_names = np.unique(y), filled=True)
```

决策树

对casual的预测(训练MAE=2.569622;测试MAE=22.440401)

对registered的预测(训练MAE=18.142090;测试MAE=116.650192)

K-近邻(训练误差)

对casual的预测: R方=0.825364;MAE=9.162251

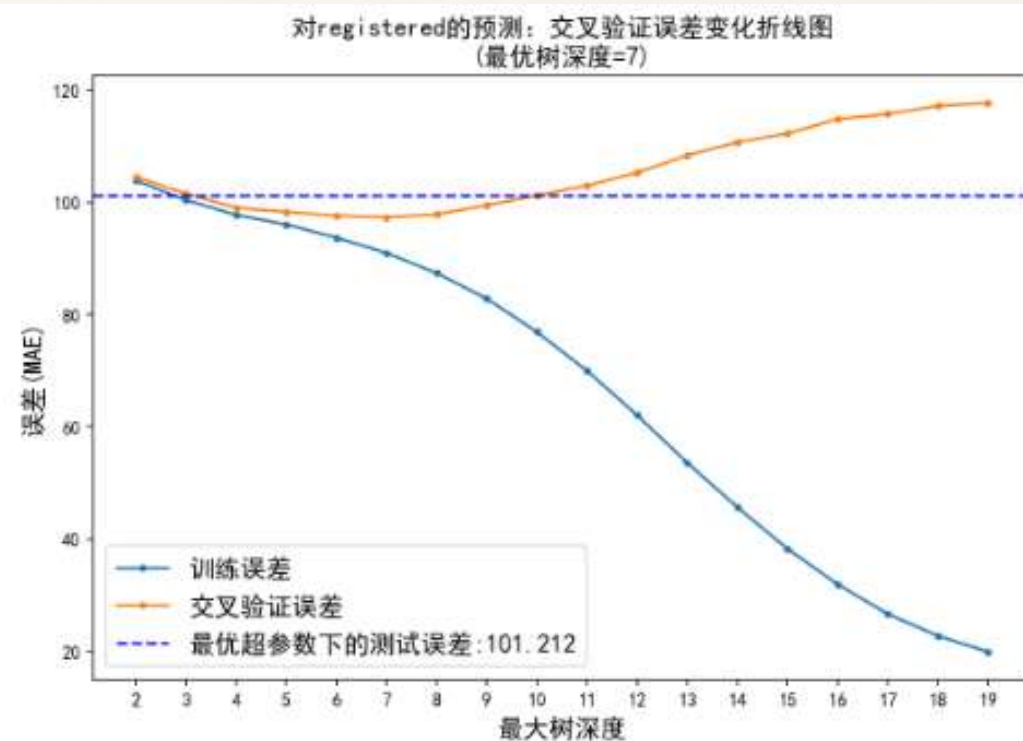
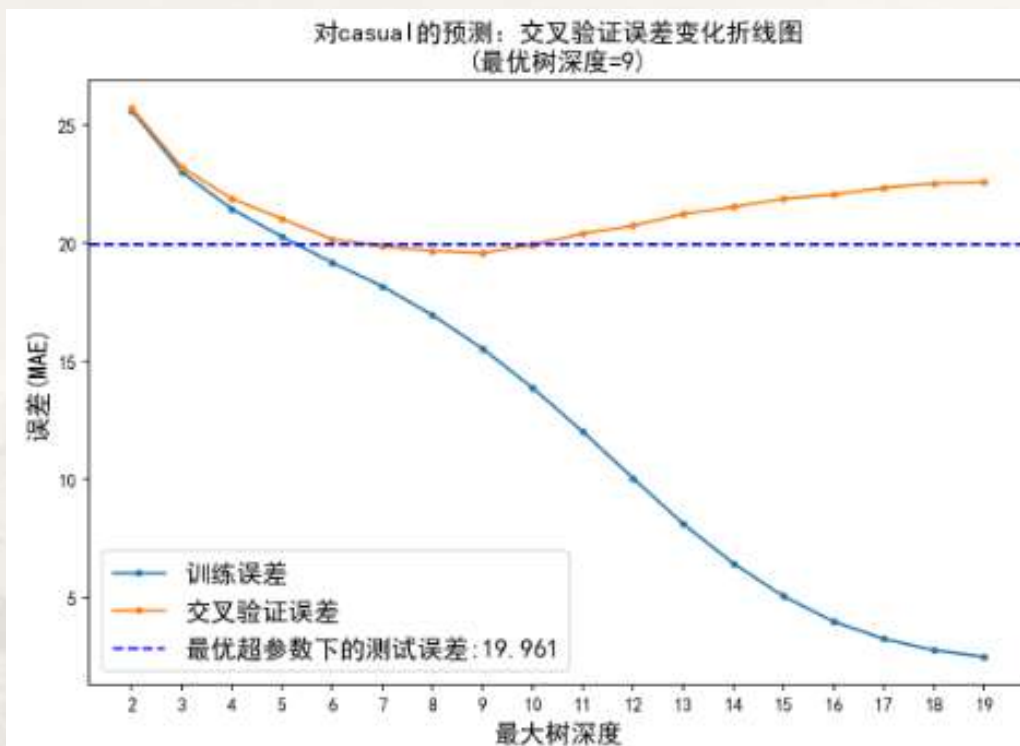
对registered的预测: R方=0.731612;MAE=41.564949

- 问:
 - 为什么测试误差仍不理想?
 - 可能是过拟合的
 - 决策树的超参数是?

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 最大树深度的超参数调优：避免过拟合！

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
for l, Y in zip([0, 1], ['casual', 'registered']):
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    trainErr = []
    valErr = []
    Deep = np.arange(2, 20)
    for d in Deep:
        modelDTC = tree.DecisionTreeRegressor(max_depth = d, random_state = 123)
        scores = cross_validate(modelDTC, X_train, y_train, scoring = 'neg_mean_absolute_error', cv = 5, return_train_score = True)
        trainErr.append(-scores['train_score'].mean())
        valErr.append(-scores['test_score'].mean())
    bestDeep = Deep[valErr.index(np.min(valErr))]
    modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
    modelDTC.fit(X_train, y_train)
    testMAE = mean_absolute_error(y_test, modelDTC.predict(X_test))
    axes[l].plot(Deep, trainErr, marker = '.', label = '训练误差')
    axes[l].plot(Deep, valErr, marker = '.', label = '交叉验证误差')
    axes[l].axhline(y = testMAE, color = 'blue', linestyle = '-', label = '最优超参数下的测试误差: {:.3f}'.format(testMAE))
    axes[l].set_title('对%s的预测：交叉验证误差变化折线图\n(最优树深度=%d)' % (Y, bestDeep), fontsize = 14)
    axes[l].set_xticks(Deep)
    axes[l].set_xlabel('最大树深度', fontsize = 14)
    axes[l].set_ylabel('误差(MAE)', fontsize = 14)
    axes[l].legend(fontsize = 14)
```

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 最大树深度的超参数调优：避免过拟合！



- 超参数调优后两个预测的测试误差均有所降低
- 问：
 - 这里超参数调优的本质？
 - 限制树生长—预修剪

调优前的决策树

对casual的预测(训练MAE=2.569622;测试MAE=22.440401)

对registered的预测(训练MAE=18.142090;测试MAE=116.650192)

决策树的预修剪

- 通过预设参数值限制树生长，称为对决策树做**预修剪** (pre-pruning)
 - 可事先指定决策树的**最大树深度**
 - 到达指定深度后就不再继续生长
 - 可事先指定节点的**最小样本量**
 - 节点样本量不应低于最小值，否则相应节点将不能继续分枝
 - Python中不指定最大树深度时默认该原则
 - 可事先指定相邻节点中输出变量**异质性下降的最小值**
 - 异质性下降不应低于最小值，否则相应节点将不能继续分枝

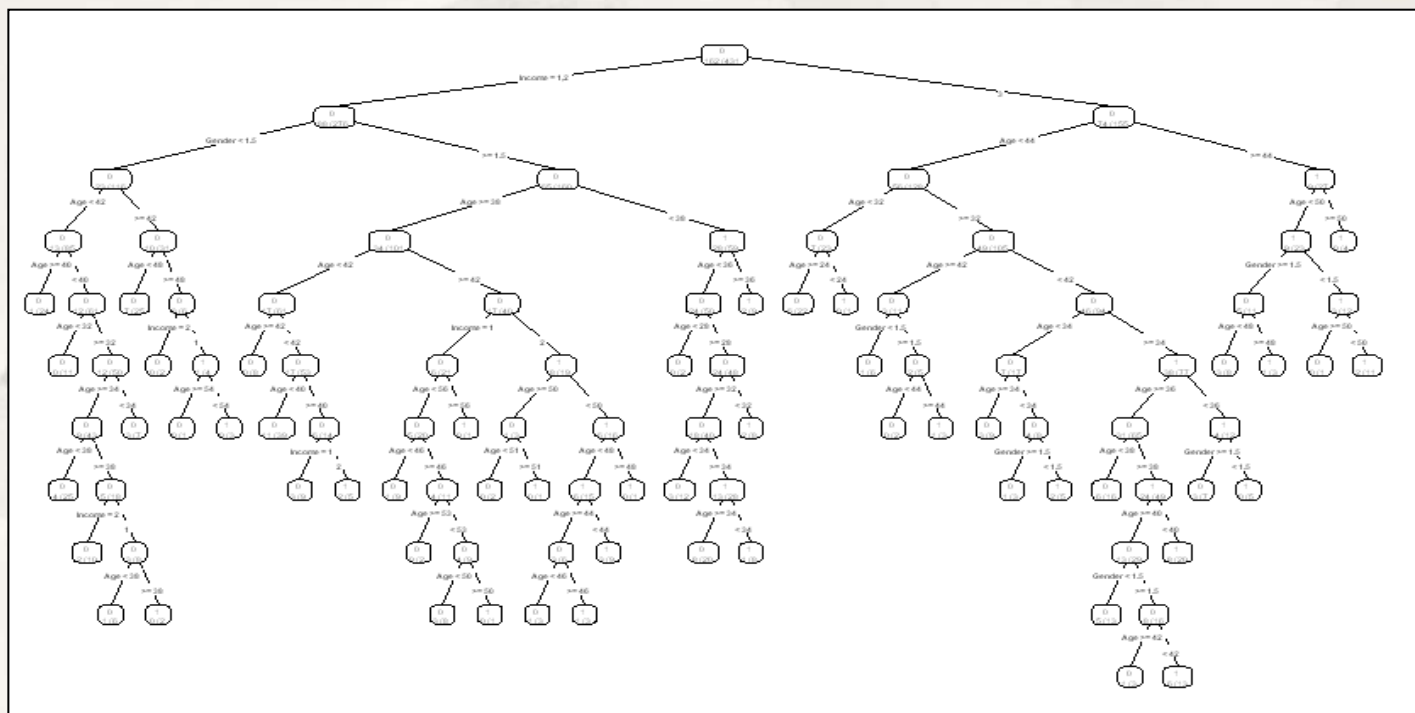
• 问：

- 预修剪**可能存在的问题**？
 - 后剪枝

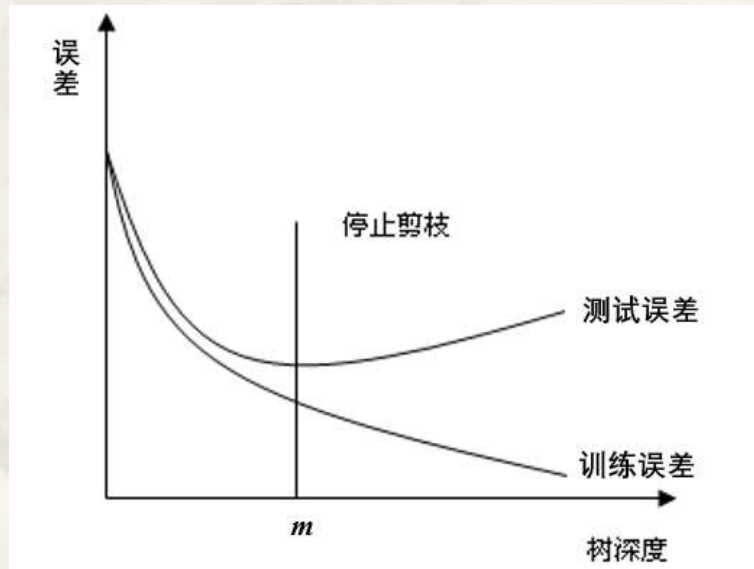
$$\Delta S^2(t) = S^2(t) - \left[\frac{N_{t_{left}}}{N_t} S^2(t_{left}) + \frac{N_{t_{right}}}{N_t} S^2(t_{right}) \right]$$

决策树的后剪枝

- 对所得的决策树，按照从叶节点向根节点的方向，逐层剪掉某些节点分枝。相对于预剪枝，这里的剪枝称为**后剪枝** (post-pruning)



- 一般原则



最小代价复杂度剪枝法

- 最小代价复杂度剪枝法 (Minimal Cost Complexity Pruning, MCCP)
 - 构建一个综合度量树优劣(偏差和方差均不大)的指标, 并依此修剪树: 代价复杂度
- 代价复杂度和最小代价复杂度

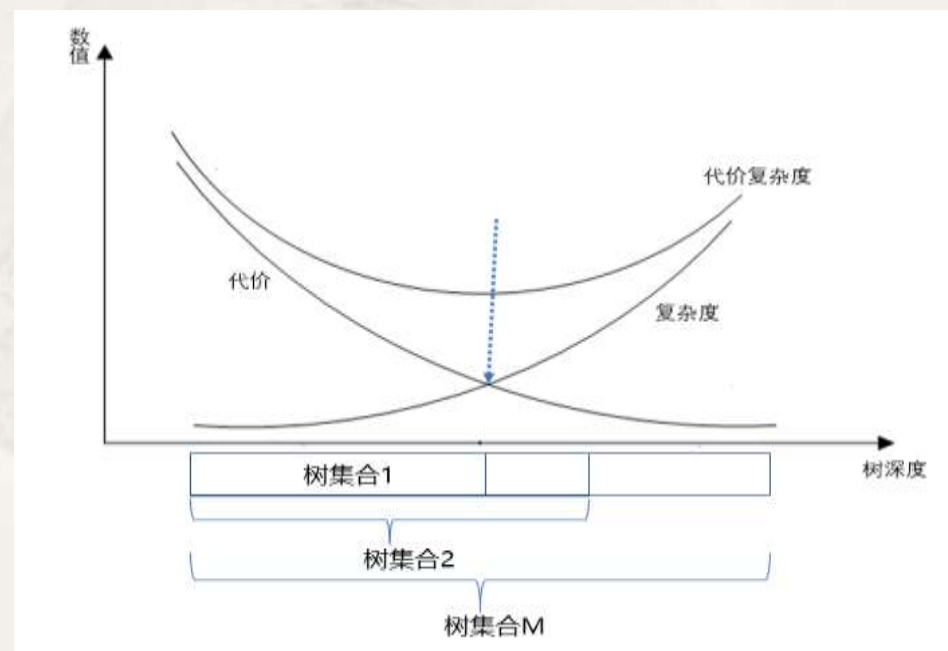
$$R_{\alpha}(T) = R(T) + \alpha|T|$$

- $R_{\alpha}(T)$ 是 α 的函数, α 为CP (Complexity Parameter) 参数

例如: 对于回归树 T

$$R_{\alpha}(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha|T|$$

- 问:
 - 代价和复杂度的关系是怎样的?
 - 最小代价复杂度意味着什么?
 - 树模型复杂度适中, 兼顾偏差和方差

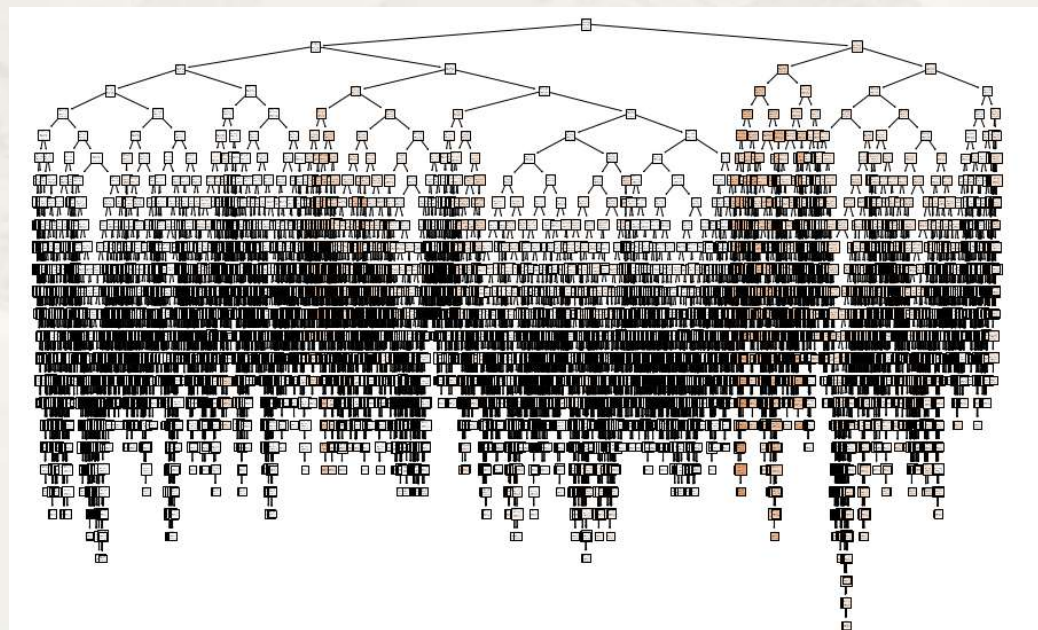


最小代价复杂度剪枝法

- 最小代价复杂度剪枝法 (Minimal Cost Complexity Pruning, MCCP)
 - 构建一个综合度量树优劣(偏差和方差均不大)的指标, 并依此修剪树: 代价复杂度
- 代价复杂度和最小代价复杂度

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

- $R_{\alpha}(T)$ 是 α 的函数, α 为CP (Complexity Parameter) 参数
- 问:
 - 你认为右图过拟合树形成的可能原因?
 - $R_{\alpha}(T)$ 主要由哪项决定?
 - 如何改进?
 - α 是模型参数吗? α 取不同值会怎样?
 - 依据最小代价复杂度, α 很小($\rightarrow 0$)会怎样? α 很大($\rightarrow \infty$)会怎样?



最小代价复杂度剪枝法

- 后剪枝过程

- 中间节点 $\{t\}$ 的代价复杂度 $R_\alpha(\{t\})$ ，定义为减掉其所有子树 T_t 后的代价复杂度：

- $R_\alpha(\{t\}) = R(\{t\}) + \alpha$

- 中间节点 $\{t\}$ 的子树 T_t 的代价复杂度 $R_\alpha(T_t)$ 定义为：

- $R_\alpha(T_t) = R(T_t) + \alpha|T_t|$

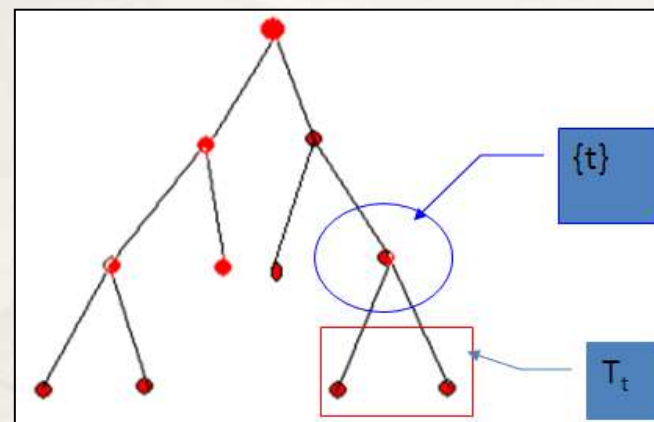
- 基于最小代价复杂度原则：

- 如果 $R_\alpha(T_t) < R_\alpha(\{t\})$ ，则应该保留子树 T_t ，此时：

- $\alpha < \frac{R(\{t\}) - R(T_t)}{|T_t| - 1}$ 成立

- 如果 $\alpha \geq \frac{R(\{t\}) - R(T_t)}{|T_t| - 1}$ ，并导致 $R_\alpha(T_t) \geq R_\alpha(\{t\})$ ，应剪掉子树 T_t

- 可见， $\frac{R(\{t\}) - R(T_t)}{|T_t| - 1}$ 越小且小于当前 α ，说明子树 T_t 对降低误差的贡献小应剪掉



- 后剪枝过程的核心目标：

- 通过超参数 (CP是超参数) 调优获得最优子树

$$R_{\alpha}(\{t\}) = R(\{t\}) + \alpha$$

$$R_{\alpha}(T_t) = R(T_t) + \alpha|T_t|$$

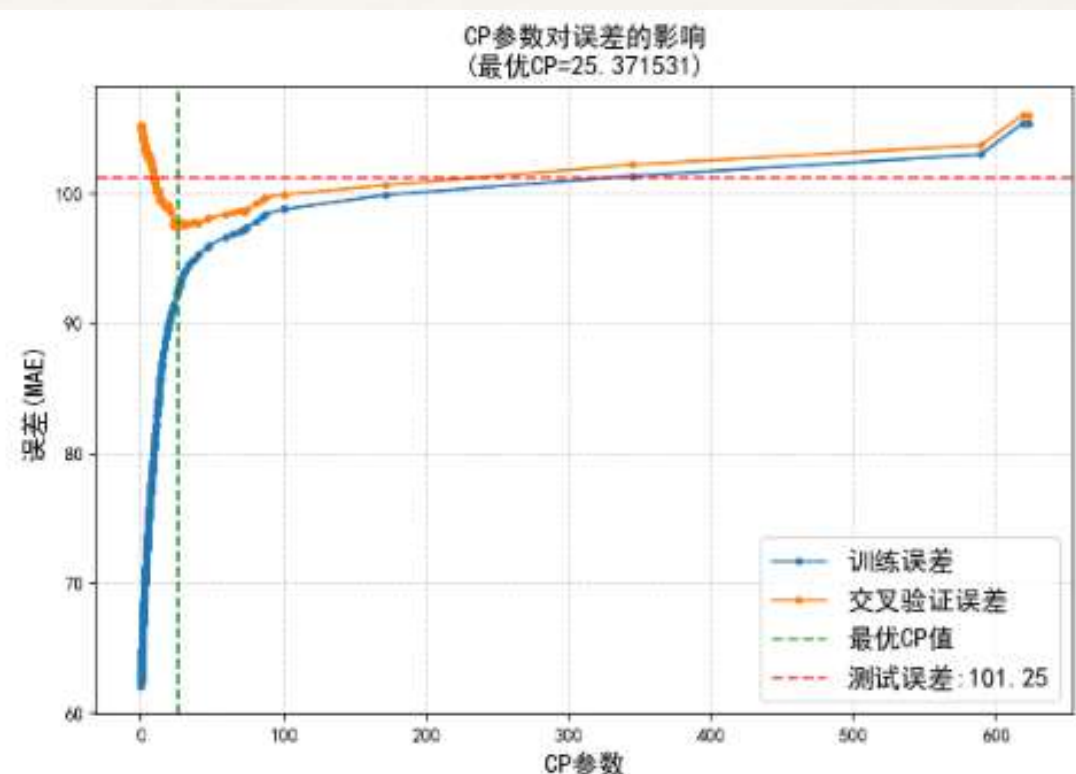
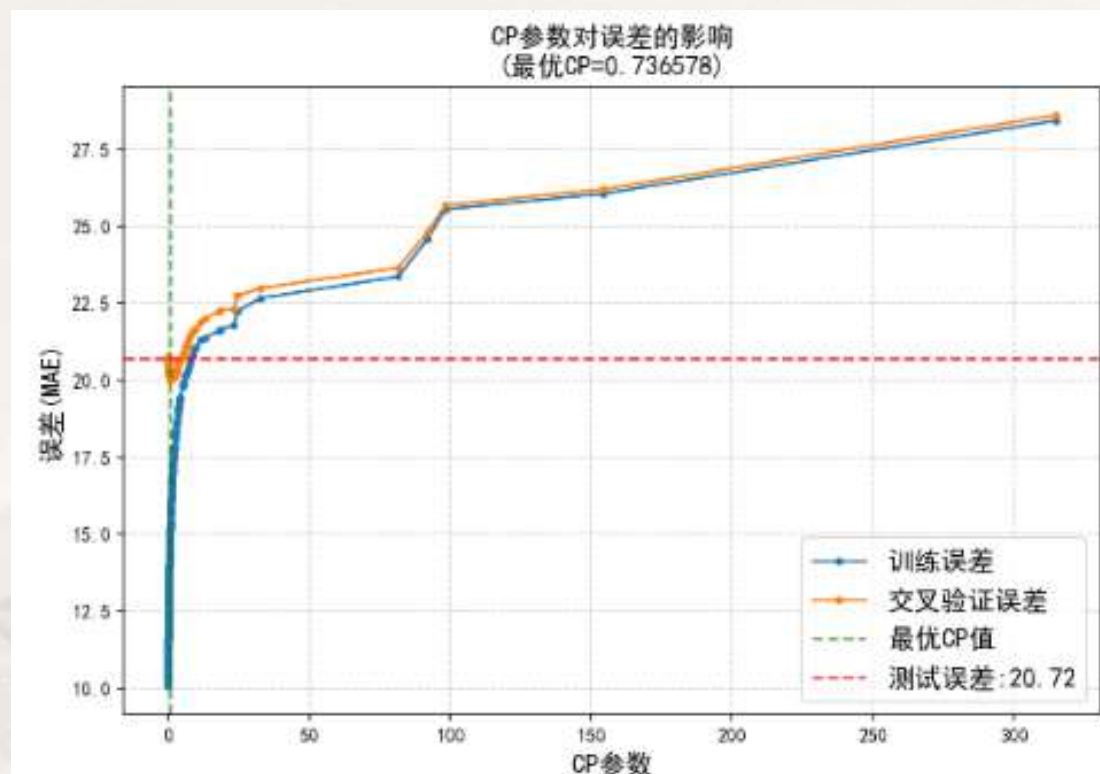
- CP超参数 α 的调优过程：逐渐增大 α 取值并依据最小代价复杂度原子剪枝，得到 K 个当前最优子树

- 开始时 $\alpha = \alpha_1 = 0$ ，当前最优子树为未进行任何剪枝的最大树，记为 T_{α_1}
- 逐渐增大CP参数 (加大复杂度惩罚)， $R_{\alpha}(\{t\})$ 和 $R_{\alpha}(T_t)$ 会随之同时增大
 - 若当下 $R(T_t) \ll R(\{t\})$ 使得 $\alpha < \frac{R(\{t\}) - R(T_t)}{|T_t| - 1}$ ，就不能剪掉子树 T_t
- 继续增大CP参数 α 的取值。当CP参数 α 从 α_1 经过若干步长增大至 α_2 时：
 - $R_{\alpha}(\{t\}) \leq R_{\alpha}(T_t)$ ，即子树 T_t 的代价复杂度开始大于 $\{t\}$ 时，应剪掉子树 T_t ，得到一个“次茂盛”的树，记为 T_{α_2}
- 重复上述步骤，直到树只剩下一个根节点为止
 - $\alpha_1 < \alpha_2 < \alpha_3 \dots < \alpha_K$ ——对应 $T_{\alpha_1}, T_{\alpha_2}, \dots, T_{\alpha_K}$ ，叶节点个数依次减少， T_{α_K} 只有根节点
- 选择验证误差最小时CP参数对应的最优子树作为最终的最优子树 T_{opt}

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 后剪枝策略中CP的超参数调优

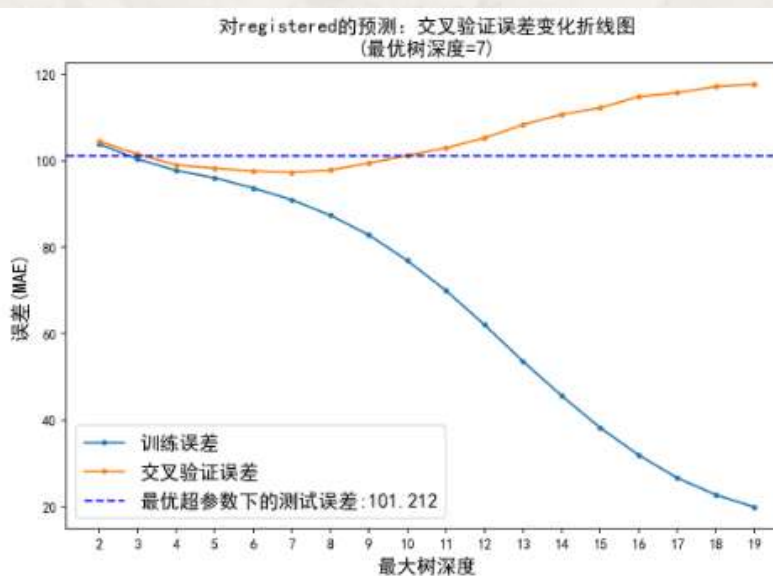
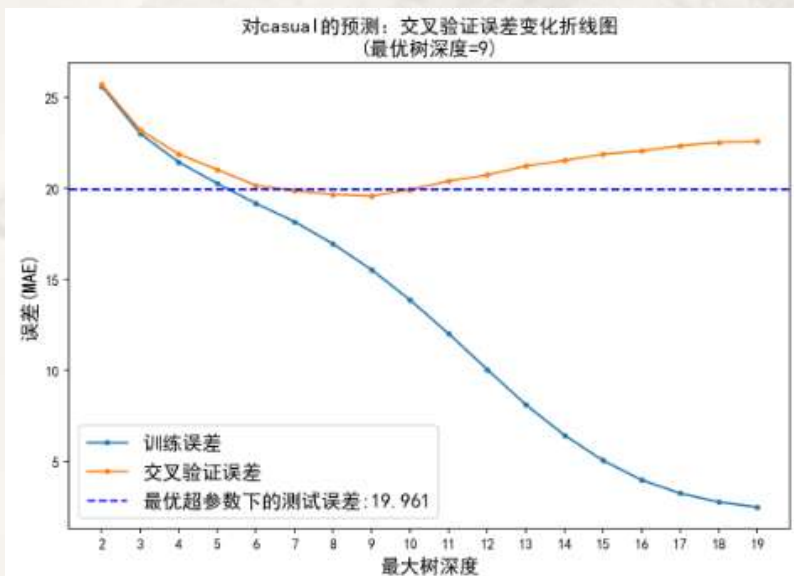
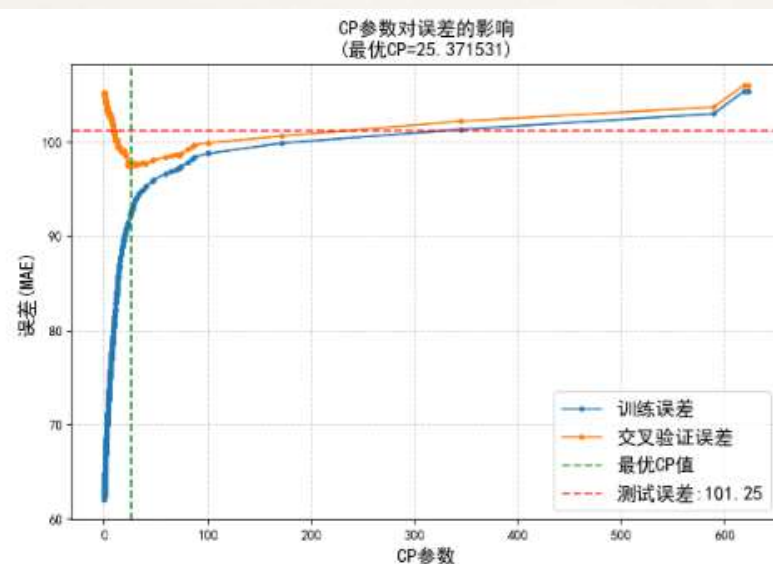
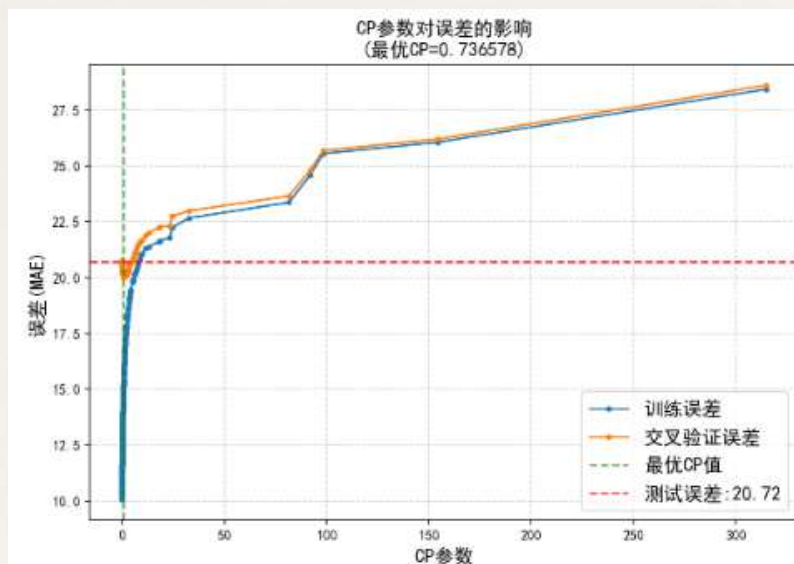
```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
for l, Y in zip([0, 1], ['casual', 'registered']):
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    trainErr = []
    valErr = []
    modelDTC = tree.DecisionTreeRegressor(max_depth = 12, random_state = 123)
    path = modelDTC.cost_complexity_pruning_path(X_train, y_train)
    alphas = path.ccp_alphas[1:-1] # 去掉完全不剪枝和全剪枝的情况
    ccp_alphas = [a for a in alphas if a > 0]
    for alpha in ccp_alphas:
        modelDTC = tree.DecisionTreeRegressor(max_depth = 12, random_state = 123, ccp_alpha = alpha)
        scores = cross_validate(modelDTC, X_train, y_train, scoring = 'neg_mean_absolute_error', cv = 5, return_train_score = True)
        trainErr.append(-scores['train_score'].mean())
        valErr.append(-scores['test_score'].mean())
    bestCP = ccp_alphas[valErr.index(np.min(valErr))]
    modelDTC = tree.DecisionTreeRegressor(max_depth = 12, random_state = 123, ccp_alpha = bestCP)
    modelDTC.fit(X_train, y_train)
    testMAE = mean_absolute_error(y_test, modelDTC.predict(X_test))
    axes[l].plot(ccp_alphas, trainErr, marker='.', label='训练误差')
    axes[l].plot(ccp_alphas, valErr, marker='.', label='交叉验证误差')
    axes[l].axvline(x = bestCP, color = 'green', linestyle = '-', alpha = 0.7, label = '最优CP值')
    axes[l].axhline(y = testMAE, color = 'r', linestyle = '-', alpha = 0.7, label = '测试误差: {:.2f}'.format(testMAE))
    axes[l].set_title('CP参数对误差的影响\n(最优CP=%f)' % bestCP, fontsize = 14)
    axes[l].grid(True, linestyle='-', alpha = 0.6)
    axes[l].set_xlabel('CP参数', fontsize = 14)
    axes[l].set_ylabel('误差(MAE)', fontsize = 14)
    axes[l].legend(fontsize = 14)
```

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 后剪枝策略中CP的超参数调优



- 问：
 - 训练误差随CP增大而上升的原因是什么？
 - 交叉验证误差先下降后上升的原因是什么？

案例二的分析：一般线性回归模型→K-近邻法→决策树



- 问:
- 对比预剪枝和后剪枝, 你倾向采纳哪个?
 - 预剪枝
 - 与K-近邻的对比
 - 对比测试误差(课后自行比较, 判断分类变量引入的作用)

K-近邻(训练误差)

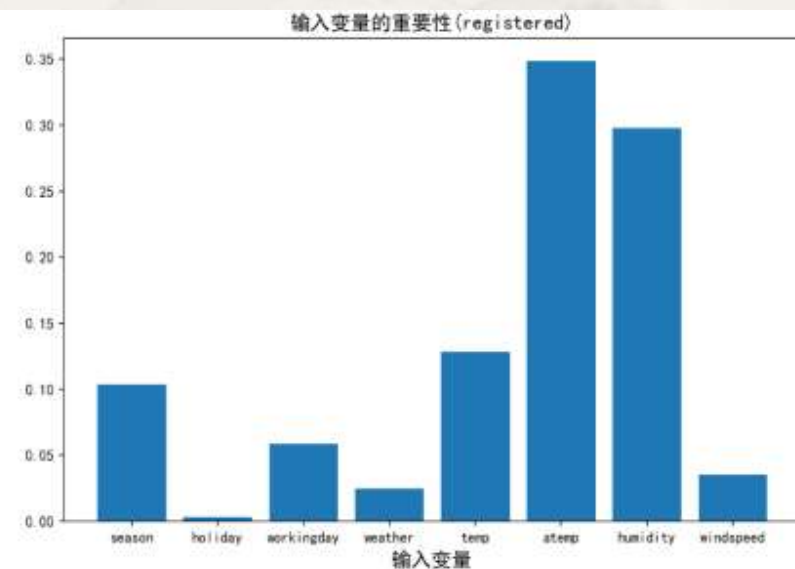
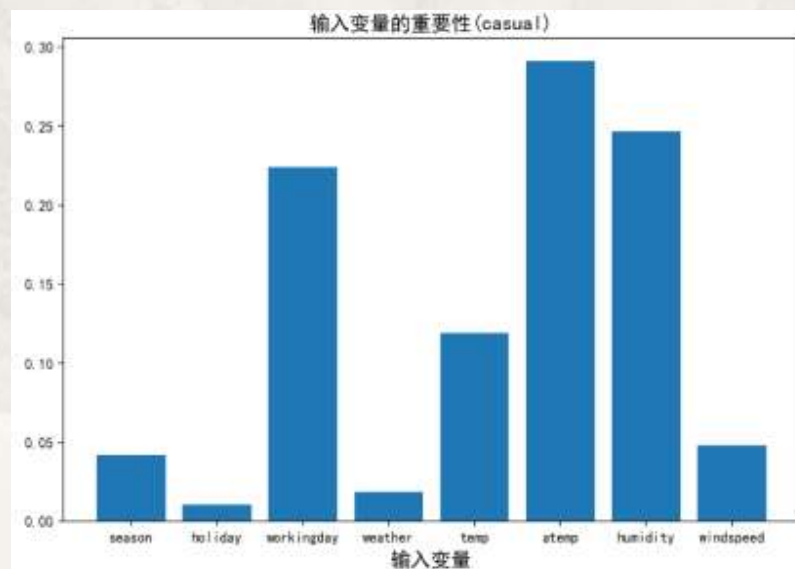
对casual的预测: R方=0.825364; MAE=9.162251

对registered的预测: R方=0.731612; MAE=41.564949

- 案例二的分析：一般线性回归模型→K-近邻法→决策树
 - 影响两类骑行日租赁量的重要因素是什么？

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
for l, bestDeep, Y in zip([0, 1], [9, 7], ['casual', 'registered']):
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
    modelDTC.fit(X_train, y_train)
    axes[l].bar(np.arange(8), modelDTC.feature_importances_)
    axes[l].set_title('输入变量的重要性(%s)' % (Y), fontsize=14)
    axes[l].set_xlabel('输入变量', fontsize=14)
    axes[l].set_xticks(np.arange(8))
    axes[l].set_xticklabels(X.columns, fontsize=10)
```

- 问：
 - 这里的特征重要性用什么测度？
 - 异质性下降



• 案例二的进一步思考：对两类日租赁量同时进行预测是否有意义？

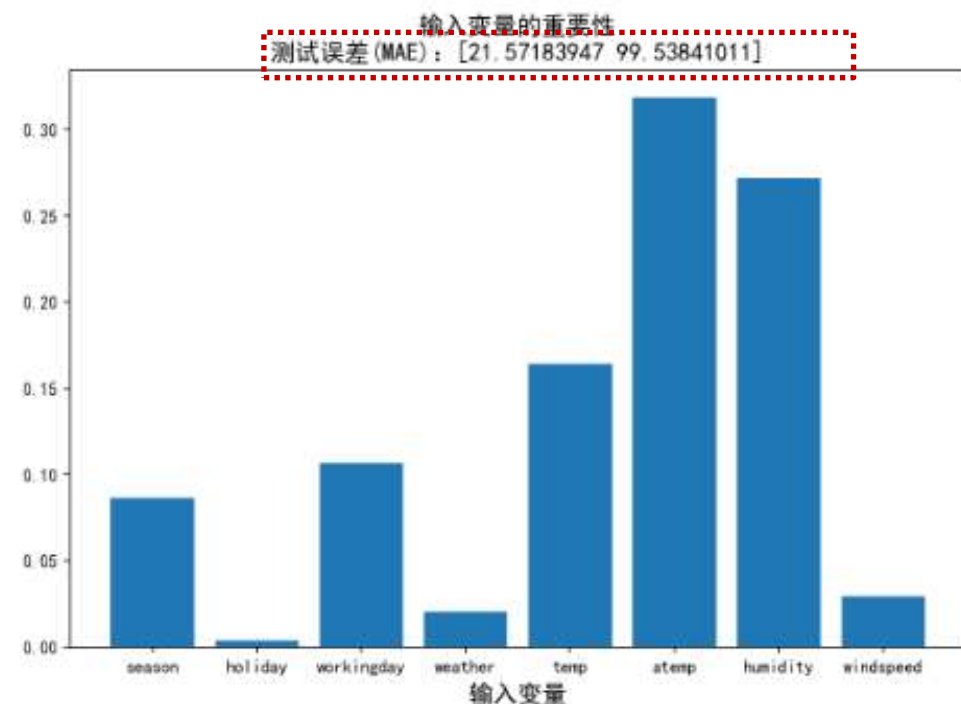
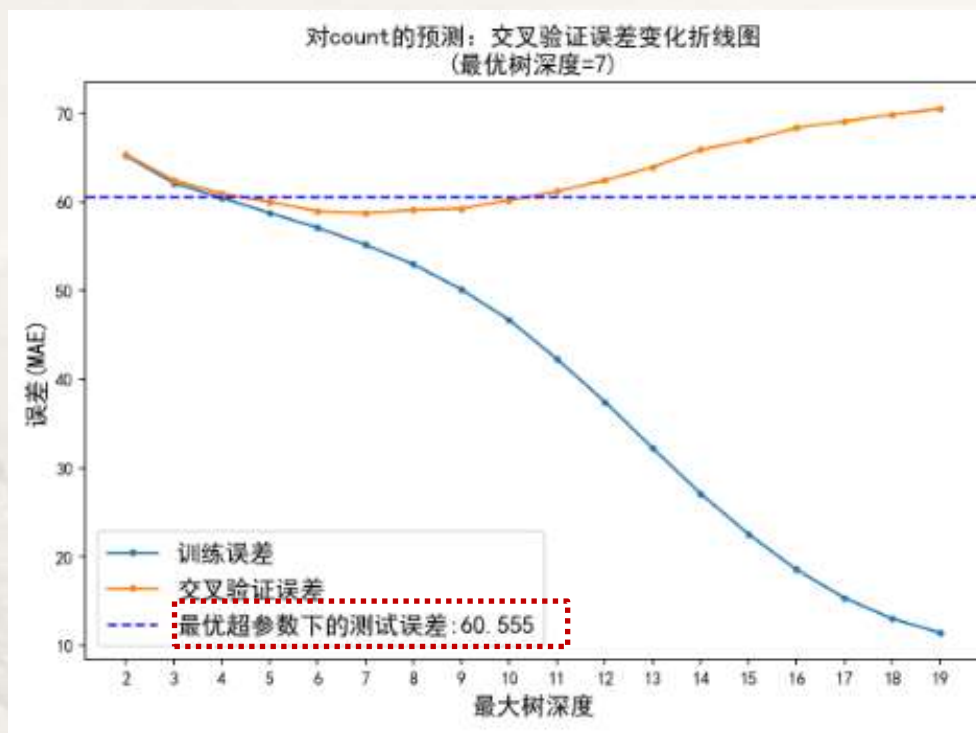
- 从管理角度，希望找到影响两种骑行日租赁量的共同天气因素，以优化运营管理
- 切入点：对两者同时进行预测
- 多输出变量的回归预测

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
y = data[['casual', 'registered']]
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
trainErr = []
valErr = []
Deep = np.arange(2, 20)
for d in Deep:
    modelDTC = tree.DecisionTreeRegressor(max_depth = d, random_state = 123)
    scores = cross_validate(modelDTC, X_train, y_train, scoring = 'neg_mean_absolute_error', cv = 5, return_train_score = True)
    trainErr.append(-scores['train_score'].mean())
    valErr.append(-scores['test_score'].mean())
bestDeep = Deep[valErr.index(np.min(valErr))]
modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
modelDTC.fit(X_train, y_train)
testMAE = mean_absolute_error(y_test, modelDTC.predict(X_test))
axes[0].plot(Deep, trainErr, marker = '.', label = '训练误差')
axes[0].plot(Deep, valErr, marker = '.', label = '交叉验证误差')
axes[0].axhline(y = testMAE, color = 'blue', linestyle = '-', label = '最优超参数下的测试误差: {:.3f}'.format(testMAE))
axes[0].set_title('对count的预测：交叉验证误差变化折线图\n(最优树深度=%d)' % (bestDeep), fontsize = 14)
axes[0].set_xticks(Deep)
axes[0].set_xlabel('最大树深度', fontsize = 14)
axes[0].set_ylabel('误差(MAE)', fontsize = 14)
axes[0].legend(fontsize = 14)

modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state=123)
modelDTC.fit(X_train, y_train)
dtrErr = mean_absolute_error(y_test, modelDTC.predict(X_test), multioutput='raw_values')
axes[1].bar(np.arange(8), modelDTC.feature_importances_)
axes[1].set_title('输入变量的重要性\n测试误差(MAE): %s' % (dtrErr), fontsize=14)
axes[1].set_xlabel('输入变量', fontsize=14)
axes[1].set_xticks(np.arange(8))
axes[1].set_xticklabels(X.columns, fontsize=10)
```

- 案例二的进一步思考：对两类日租赁量同时进行预测是否有意义？

- 从管理角度，希望找到影响两种骑行日租赁量的共同天气因素，以优化运营管理
- 切入点：对两者同时进行预测
- 多输出变量的回归预测



案例五：药物适用性研究

- 案例五：药物适用性研究—多分类的预测问题
 - 目标：探索影响药物适用性的重要因素和用药建议

- 案例五数据：

- 数据的预处理：重编码

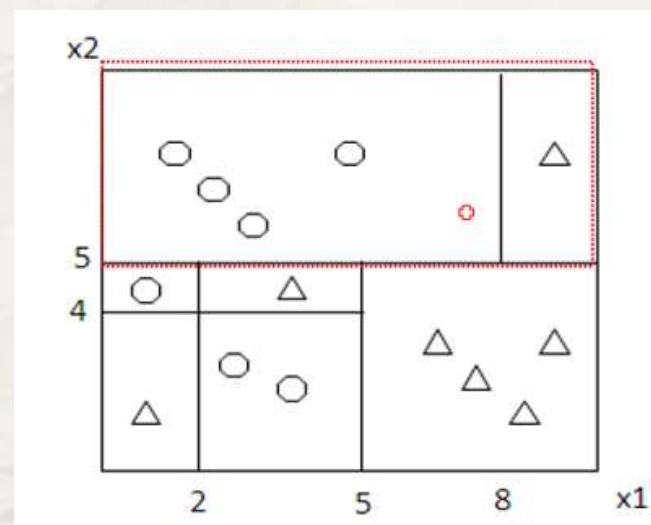
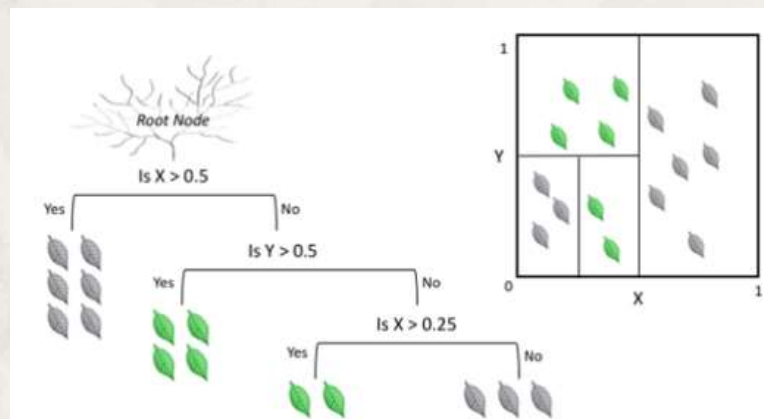
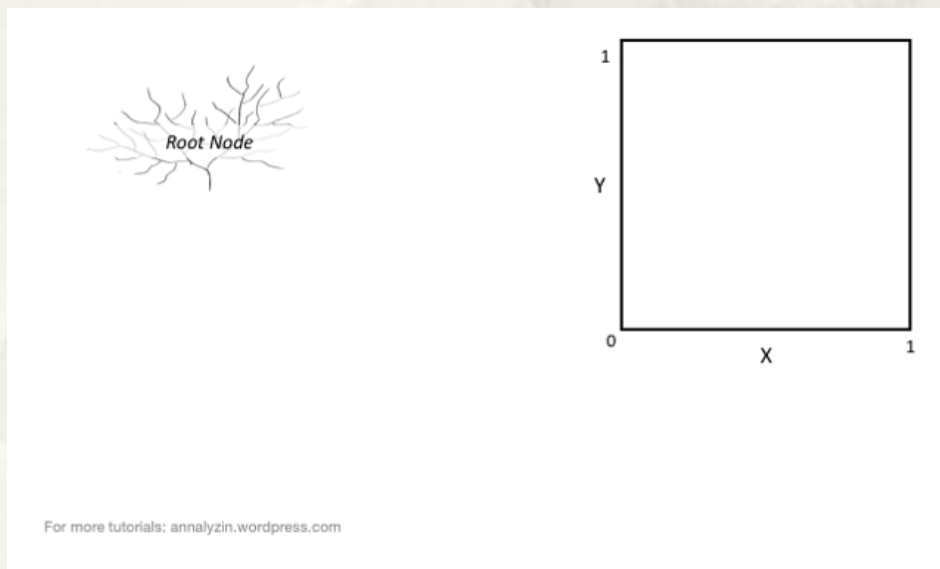
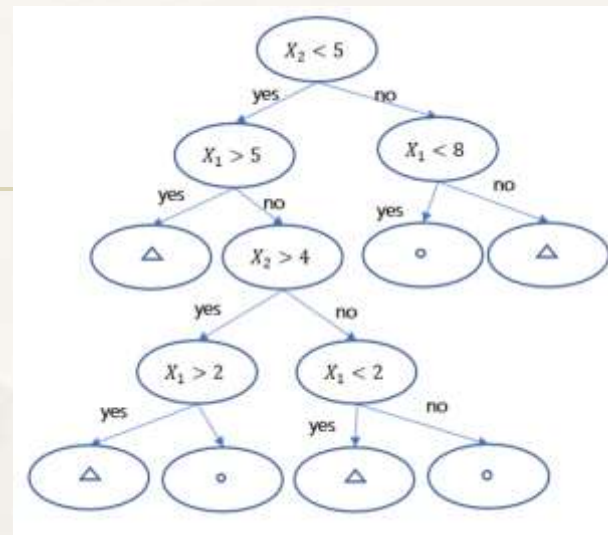
```
1 data = pd.read_csv('药物研究.txt')
2 le = LabelEncoder()
3 le.fit(data['Sex'])
4 data['SexC'] = le.transform(data['Sex'])
5 data['BPC'] = le.fit(data['BP']).transform(data['BP'])
6 data['CholesterolC'] = le.fit(data['Cholesterol']).transform(data['Cholesterol'])
7 data.head()
```

	Age	Sex	BP	Cholesterol	Na	K	Drug	SexC	BPC	CholesterolC
0	23	F	HIGH	HIGH	0.792535	0.031258	drugY	0	0	0
1	47	M	LOW	HIGH	0.739309	0.056468	drugC	1	1	0
2	47	M	LOW	HIGH	0.697269	0.068944	drugC	1	1	0
3	28	F	NORMAL	HIGH	0.563682	0.072289	drugX	0	2	0
4	61	F	LOW	HIGH	0.559294	0.030998	drugY	0	1	0

- 同时包含数值型和分类型输入变量，非常适合采用决策树！

分类预测中的决策树

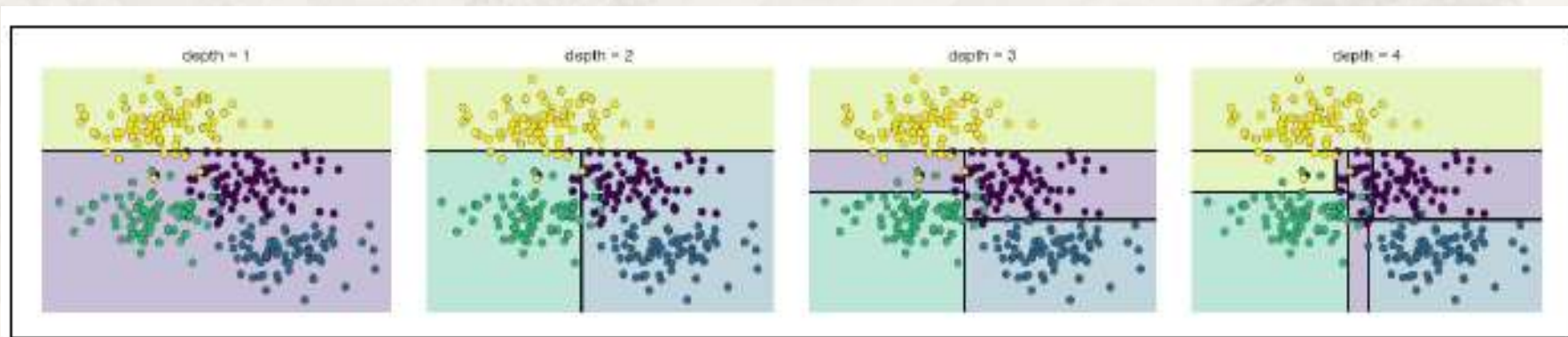
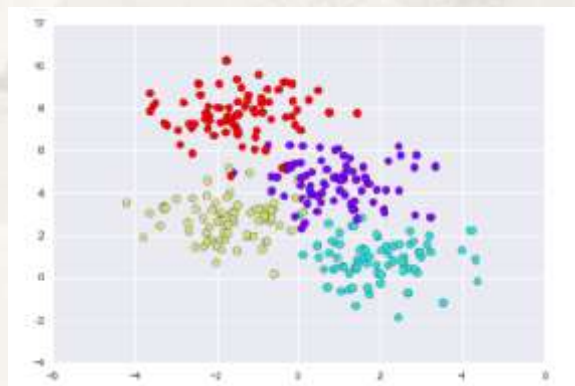
- 对 p 个输入变量构成的 p 维实数空间进行反复划分，形成多个不相交的小区域
- 每个小区域是一个数据分组，对应决策树的一个叶节点



$$f(X) = \sum_{m=1}^M c_m \cdot 1_{(X \in R_m)}$$

分类预测中的决策树

- 问：依据什么原则确定当前最佳分组变量和组限？
 - 两区域包含的样本观测点尽量纯正，异质性 (Impurity) 低
- 问：为什么？
 - 低异质性下，分类预测错判率低，推理规则的置信度高



- 分类预测中异质性度量

- 基尼 (Gini) 系数和基尼系数的下降

$$G(t) = 1 - \sum_{k=1}^K P^2(k|t) = \sum_{k=1}^K P(k|t)(1 - P(k|t))$$

- 输出变量的异质性下降:

$$\Delta G(t) = G(t) - \left[\frac{N_{t_{left}}}{N_t} G(t_{left}) + \frac{N_{t_{right}}}{N_t} G(t_{right}) \right]$$

- 问:

- 方括号中是什么?
 - 最佳分组变量和组限值使 $\Delta G(t)$ 最大还是最小呢?

- 熵和熵的下降:

$$Ent(t) = - \sum_{i=1}^K P_i \log P_i$$

熵为非负数

- $\Delta Ent(t) = Ent(t) - \left[\frac{N_{t_{left}}}{N_t} Ent(t_{left}) + \frac{N_{t_{right}}}{N_t} Ent(t_{right}) \right]$

- 问:

- 熵小或大意味着什么?
 - 熵的意义

- 熵 (Entropy) 也称信息熵。1948年香农 (C. E. Shannon) 借用热力学的热熵概念，提出用信息熵度量信息论中信息传递过程中的信源不确定性
 - 信息的价值：消除不确定性 (例：甲乙两名同学竞选班长，考试作弊)
 - 信息价值的度量：熵

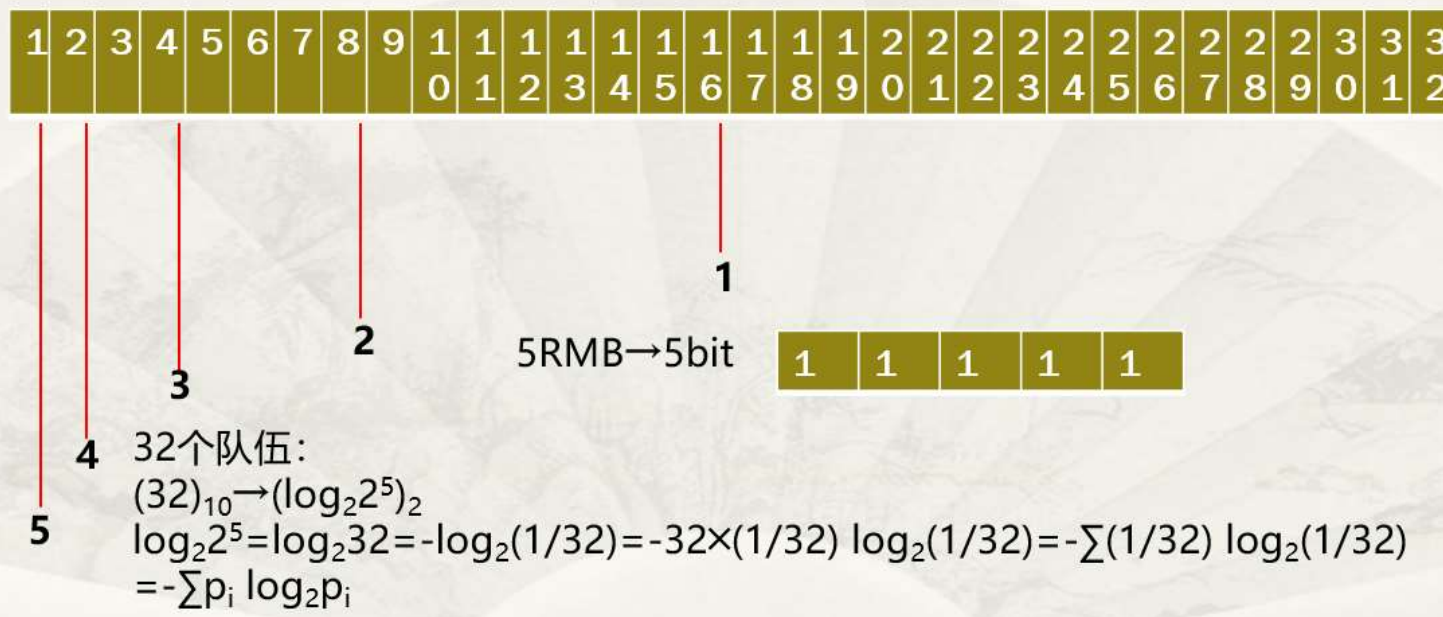
- 例如：世界杯足球赛

$p_i = 1/32, (i = 1, 2, \dots, 32)$ 完全不确定性



$p_k = 1$ 确定性

$$Ent(U) = - \sum_{i=1}^K p_i \log_2 p_i$$



- 熵采用自然对数，非负数
- $Ent(U) = 0$ 最小：没有信息发送的不确定性
- $p_i = \frac{1}{K}$ ：信息发送的不确定性最大，熵达到最大 $Ent(U) = -\log_2 \frac{1}{K}$

• 问：

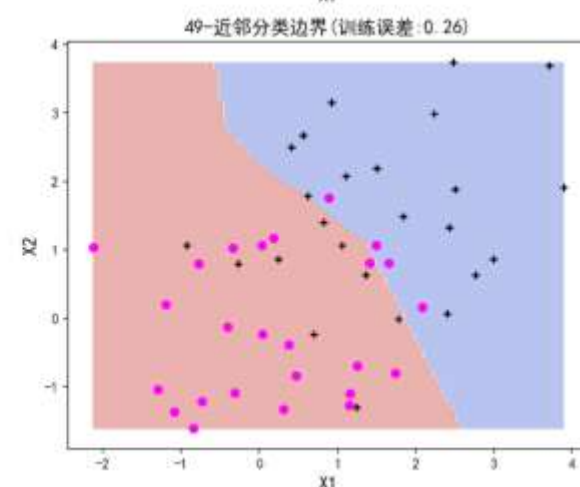
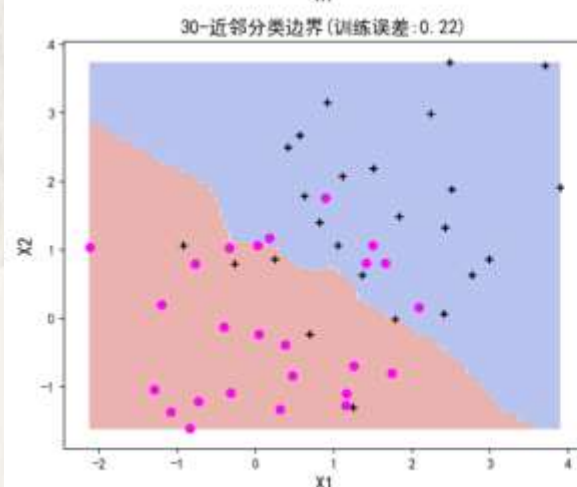
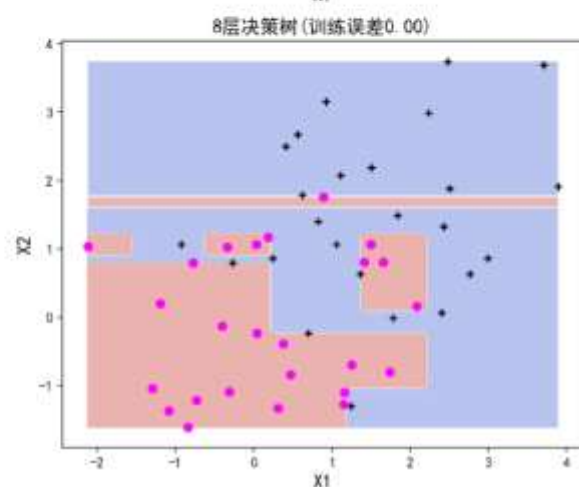
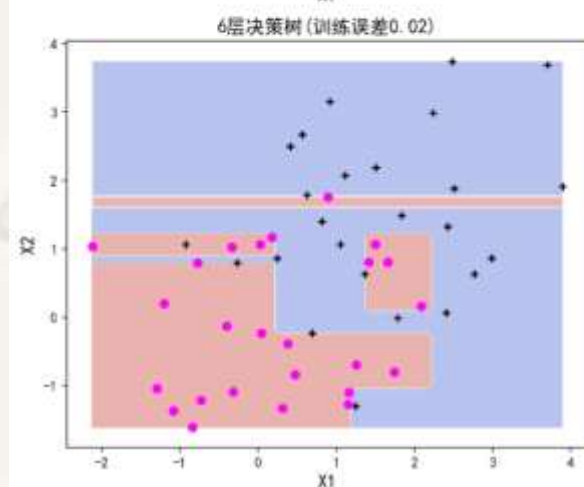
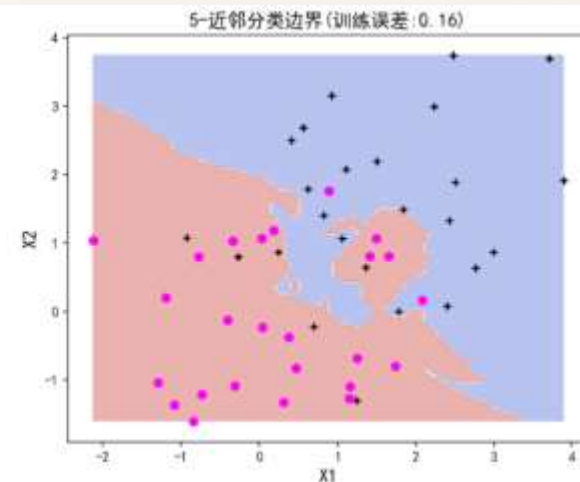
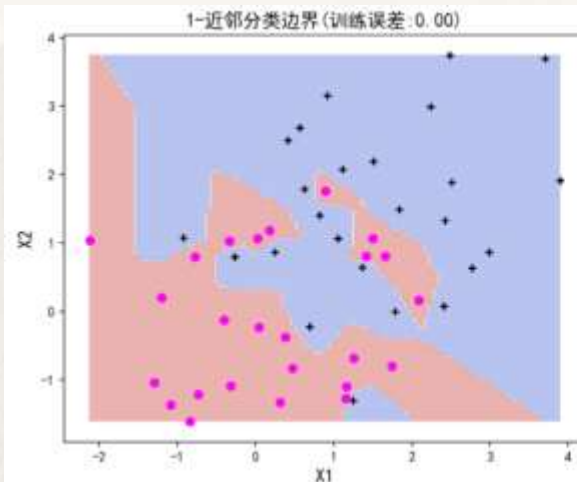
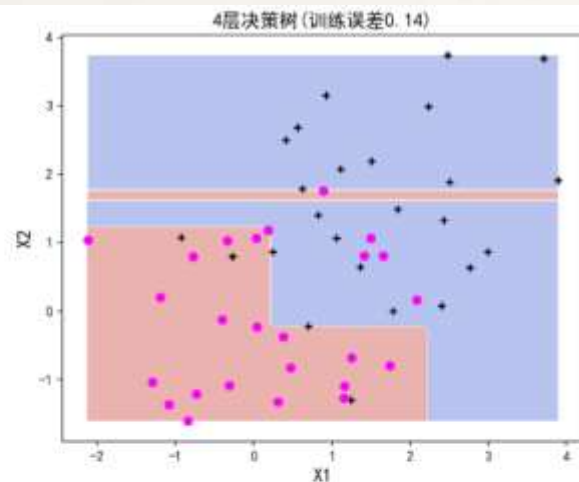
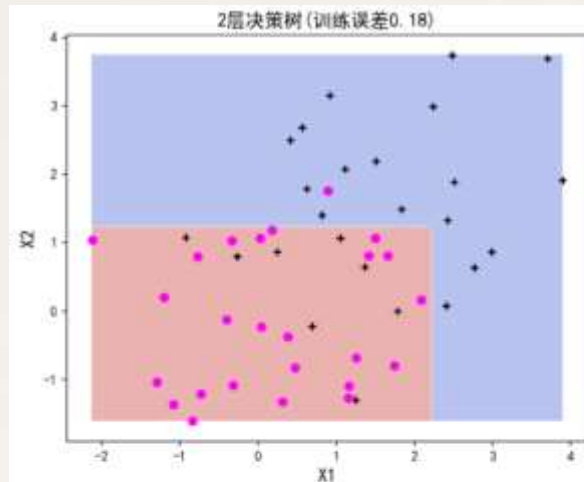
- 分类预测的分组中希望熵大些还是小些？

- 分类预测中的决策树——非线性特点
 - Python模拟和启示：认识决策树的分类边界

```
1 np.random.seed(123)
2 N = 50
3 n = int(0.5*N)
4 X = np.random.normal(0, 1, size=100).reshape(N, 2)
5 y = [0]*n + [1]*n
6 X[0:n] = X[0:n] + 1.5
7 X01, X02 = np.meshgrid(np.linspace(X[:, 0].min(), X[:, 0].max(), 100), np.linspace(X[:, 1].min(), X[:, 1].max(), 100))
8 X0 = np.hstack((X01.reshape(10000, 1), X02.reshape(10000, 1)))
9 fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 12), dpi=150)
10 for K, H, L in [(2, 0, 0), (4, 0, 1), (6, 1, 0), (8, 1, 1)]:
11     modelDTC = tree.DecisionTreeClassifier(max_depth=K, random_state=123)
12     modelDTC.fit(X, y)
13     Y0hat = modelDTC.predict(X0).reshape(X01.shape)
14     axes[H, L].contourf(X01, X02, Y0hat, alpha=0.4, cmap=plt.cm.coolwarm)
15     axes[H, L].scatter(X[:, 0], X[:, 1], color='black', marker='+')
16     axes[H, L].scatter(X[n:N, 0], X[n:N, 1], color='magenta', marker='o')
17     axes[H, L].set_title('%d层决策树(训练误差%.2f)' % ((K, 1 - modelDTC.score(X, y))), fontsize=14)
18     axes[H, L].set_xlabel('X1', fontsize=14)
19     axes[H, L].set_ylabel('X2', fontsize=14)
```

Chapter6-2.ipynb

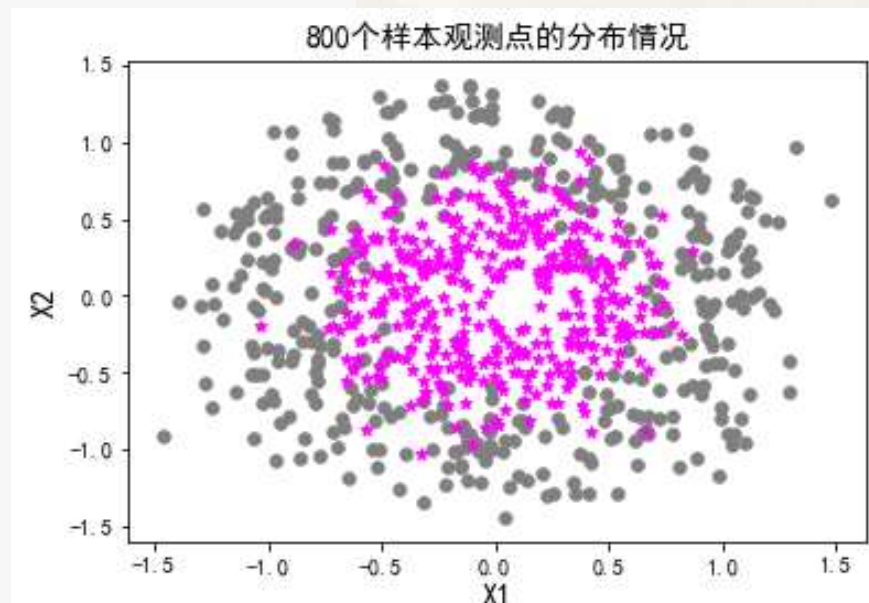
- 分类预测中的决策树——非线性特点
 - Python模拟和启示：认识决策树的分类边界



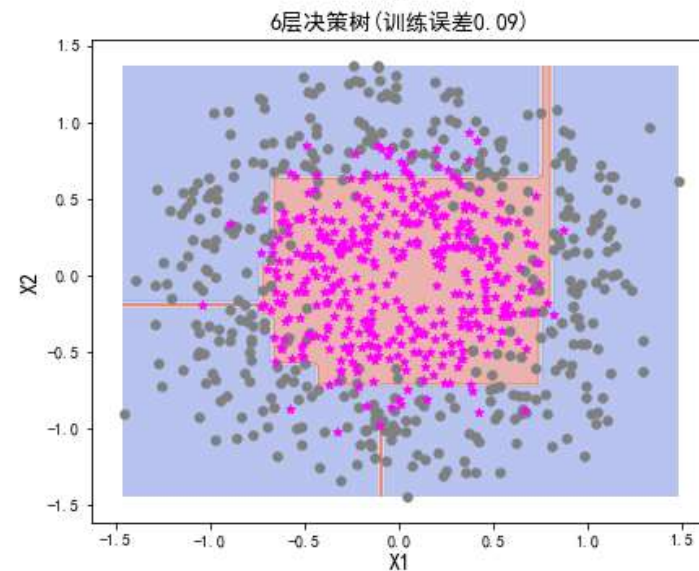
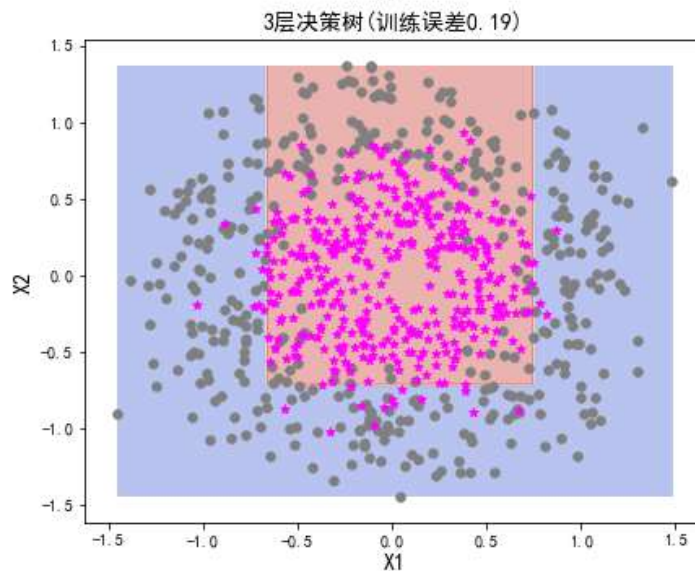
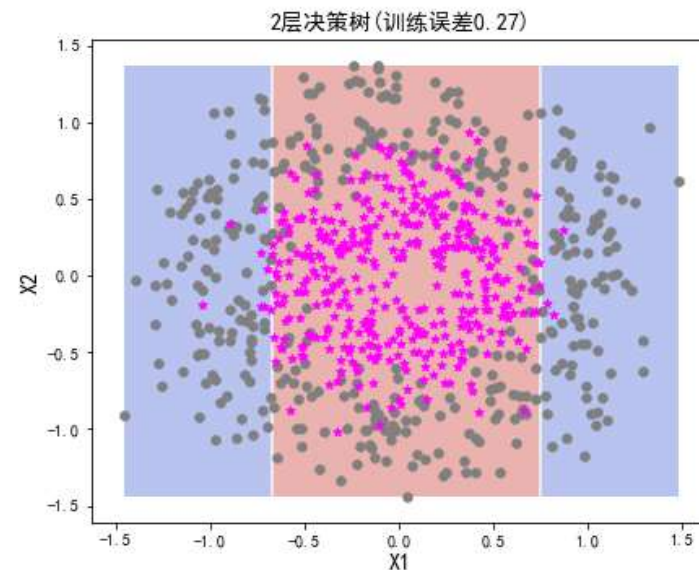
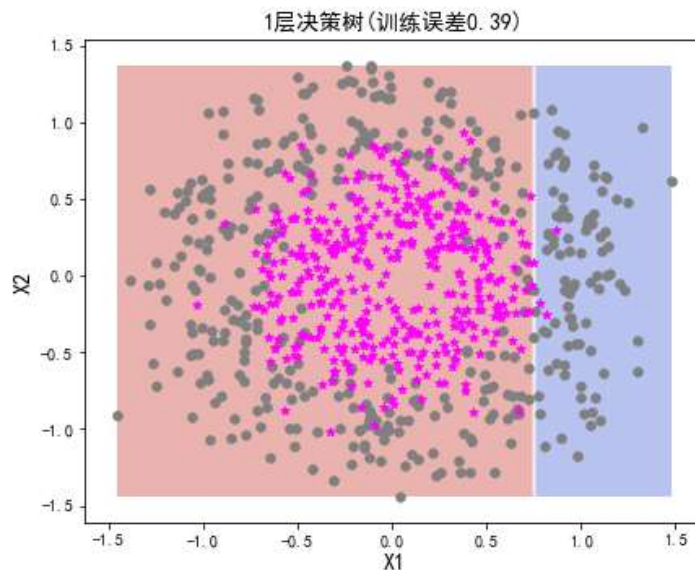
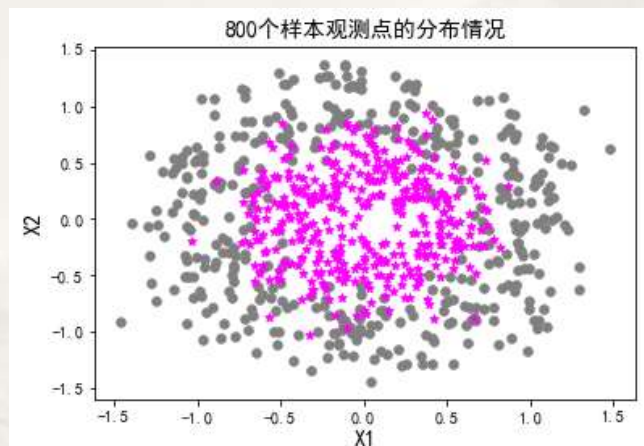
- 分类预测中的决策树——非线性特点
 - Python模拟和启示：认识决策树的分类边界

Chapter6-2.ipynb

```
np.random.seed(123)
N = 800
X, y = make_circles(n_samples=N, noise=0.2, factor=0.5, random_state=123)
X01, X02 = np.meshgrid(np.linspace(X[:, 0].min(), X[:, 0].max(), 100), np.linspace(X[:, 1].min(), X[:, 1].max(), 100))
X0 = np.hstack((X01.reshape(10000, 1), X02.reshape(10000, 1)))
colors = ['grey', 'magenta']
markers = ['o', '*']
for k, col, m in zip(set(y), colors, markers):
    plt.scatter(X[y==k, 0], X[y==k, 1], color=col, s=30, marker=m)
plt.title('%d个样本观测点的分布情况' % N, fontsize=14)
plt.xlabel('X1', fontsize=14)
plt.ylabel('X2', fontsize=14)
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 12))
for K, H, L in [(1, 0, 0), (2, 0, 1), (3, 1, 0), (6, 1, 1)]:
    modelDTC = tree.DecisionTreeClassifier(max_depth=K, random_state=123)
    modelDTC.fit(X, y)
    Y0hat = modelDTC.predict(X0).reshape(X01.shape)
    axes[H, L].contourf(X01, X02, Y0hat, alpha=0.4, cmap=plt.cm.coolwarm)
    for k, col, m in zip(set(y), colors, markers):
        axes[H, L].scatter(X[y==k, 0], X[y==k, 1], color=col, s=30, marker=m)
        axes[H, L].set_title('%d层决策树(训练误差%.2f)' % (K, 1 - modelDTC.score(X, y)), fontsize=14)
        axes[H, L].set_xlabel('X1', fontsize=14)
        axes[H, L].set_ylabel('X2', fontsize=14)
```



- 分类预测中的决策树——非线性特点
 - Python模拟和启示：认识决策树的分类边界



- 决策树可以解决非线性分类问题！

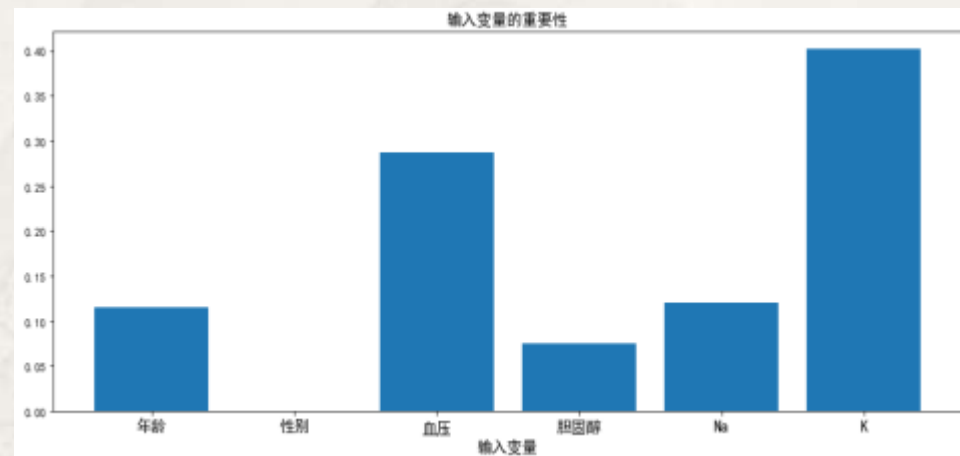
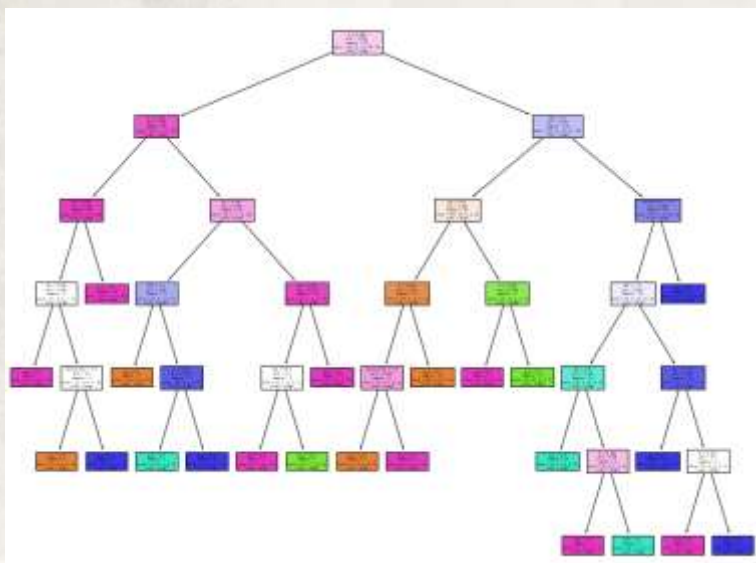
- **案例五：药物适用性研究--多分类的预测问题**
 - 目标：探索影响药物适用性的**重要因素**和**用药建议**

```
1 X = data[['Age', 'SexC', 'BPC', 'CholesterolC', 'Na', 'K']]
2 y = data['Drug']
3 modelDTC = tree.DecisionTreeClassifier(random_state=123)
4 modelDTC.fit(X, y)
5 print('模型的评价: \n', classification_report(y, modelDTC.predict(X)))
6 print('未修剪前的测试误差: %.3f' % (1 - cross_val_score(modelDTC, X, y, cv=10, scoring='accuracy')).mean())
7 plt.figure(figsize=(15, 12))
8 tree.plot_tree(modelDTC, feature_names=list(X), class_names=np.unique(y), filled=True)
```

模型的评价:

	precision	recall	f1-score	support
drugA	1.00	1.00	1.00	23
drugB	1.00	1.00	1.00	16
drugC	1.00	1.00	1.00	16
drugX	1.00	1.00	1.00	54
drugY	1.00	1.00	1.00	91
accuracy			1.00	200
macro avg	1.00	1.00	1.00	200
weighted avg	1.00	1.00	1.00	200

未修剪前的测试误差:0.120



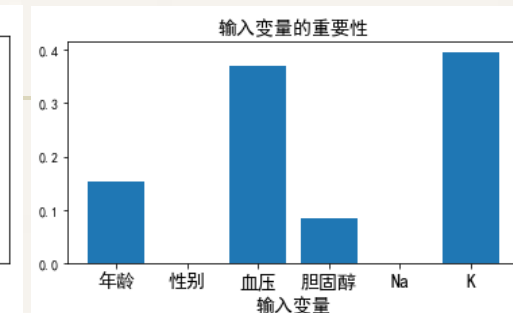
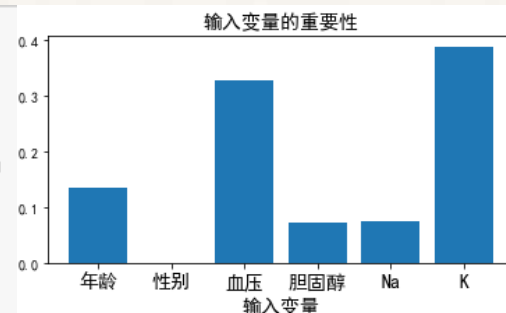
- **问：**
 - 树较为茂盛，为防止过拟合应做什么？

• 案例五的后剪枝

模型评价 (cp=0.02, 训练误差=0.085, 交叉验证误差=0.155)

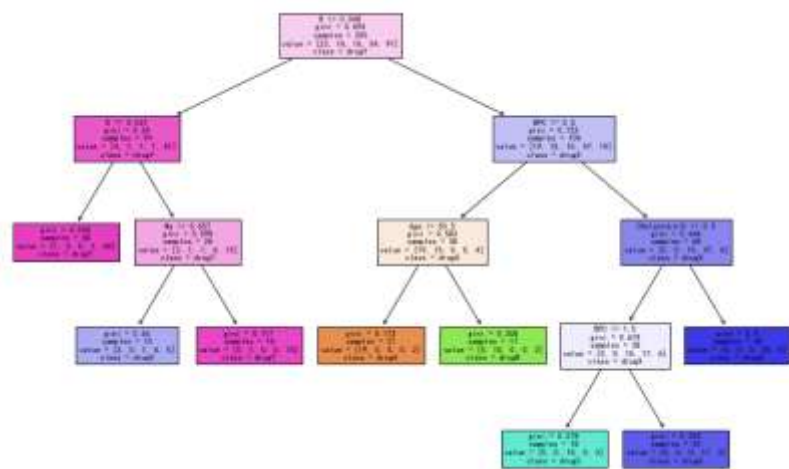
模型评价 (cp=0.05, 训练误差=0.115, 交叉验证误差=0.180)

```
for cp in [0.02, 0.05, 0.1]:
    modelDTC = tree.DecisionTreeClassifier(random_state=123, ccp_alpha=cp)
    modelDTC.fit(X, y)
    CVErr = (1-cross_val_score(modelDTC, X, y, cv=10, scoring='accuracy')).mean()
    print('模型评价 (cp=%.2f, 训练误差=%.3f, 交叉验证误差=%.3f)' % (cp, 1-modelDTC.score(X, y), CVErr))
    plt.figure(figsize=(6, 3))
    plt.bar(np.arange(6), modelDTC.feature_importances_)
    plt.title('输入变量的重要性', fontsize=14)
    plt.xlabel('输入变量', fontsize=14)
    plt.xticks(np.arange(6), ['年龄', '性别', '血压', '胆固醇', 'Na', 'K'], fontsize=14)
    plt.show()
    plt.figure(figsize=(15, 10))
    tree.plot_tree(modelDTC, feature_names=list(X), class_names=np.unique(y), filled=True)
```

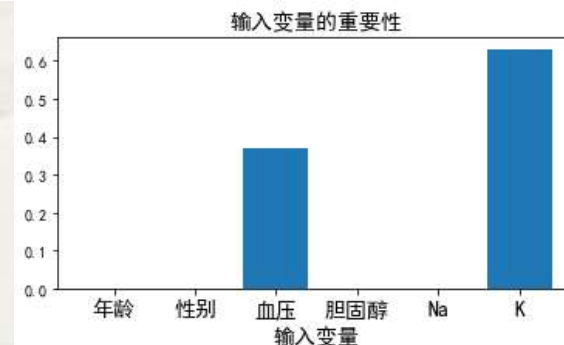
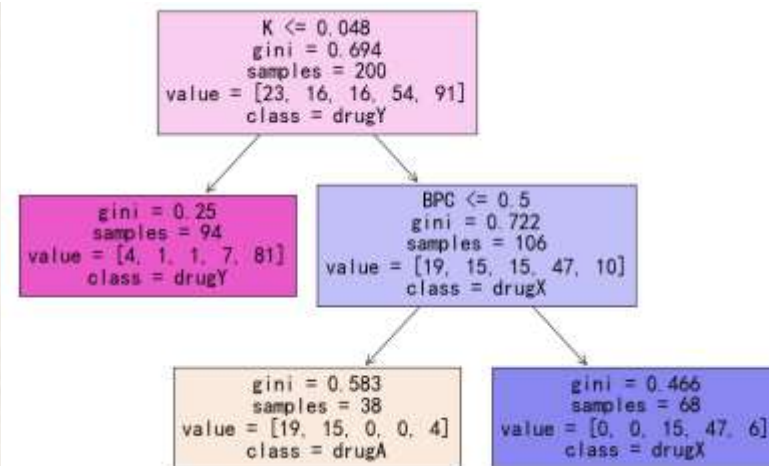
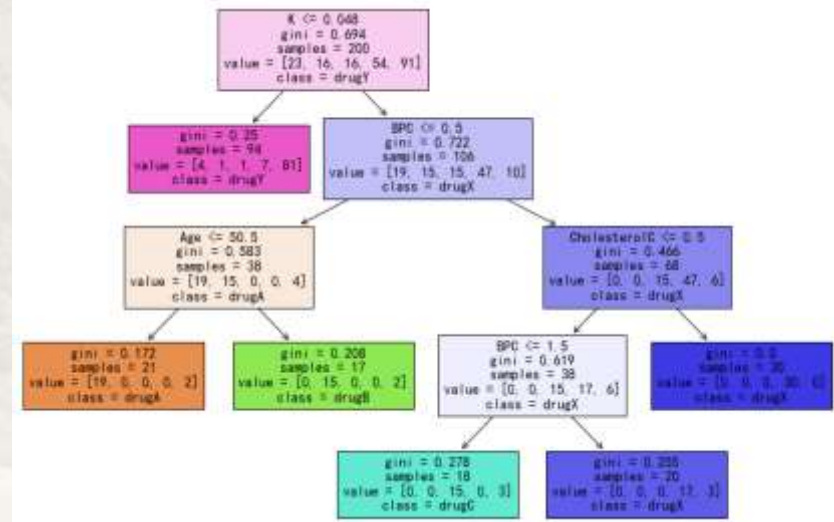


模型评价 (cp=0.10, 训练误差=0.265, 交叉验证误差=0.305)

模型评价 (cp=0.02, 训练误差=0.085, 交叉验证误差=0.155)



模型评价 (cp=0.05, 训练误差=0.115, 交叉验证误差=0.180)



• 问:

• 针对本案例这种分析思路有问题吗?

- **案例五：药物适用性研究——多分类的预测问题**
 - 目标：探索影响药物适用性的**重要因素**和**用药建议**
 - 对问题**本身的思考**
 - **直接指定最大树深度为3，合理吗？**

```
data = pd.read_csv('药物研究.txt')
le = LabelEncoder()
data['SexC'] = le.fit(data['Sex']).transform(data['Sex'])
data['BPC'] = le.fit(data['BP']).transform(data['BP'])
data['CholesterolC'] = le.fit(data['Cholesterol']).transform(data['Cholesterol'])
data['Na/K'] = data['Na']/data['K']
data.head()
```

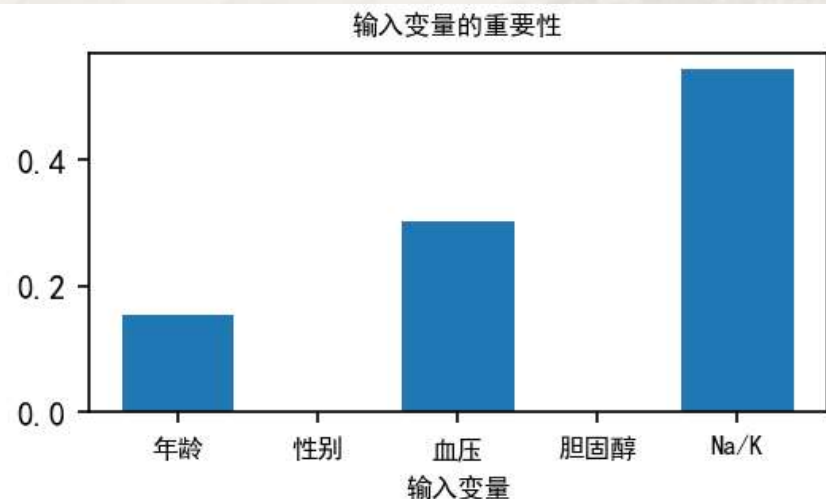
```
X = data[['Age', 'SexC', 'BPC', 'CholesterolC', 'Na/K']]
y = data['Drug']
modelDTC = tree.DecisionTreeClassifier(max_depth = 3, random_state = 123)
modelDTC.fit(X, y)
print('模型评价: \n', classification_report(y, modelDTC.predict(X)))
print('输入变量重要性: \n', modelDTC.feature_importances_)
plt.figure(figsize = (4, 2), dpi = 150)
plt.bar(np.arange(5), modelDTC.feature_importances_)
plt.xticks(np.arange(5), ['年龄', '性别', '血压', '胆固醇', 'Na/K'], fontsize = 8)
plt.title('输入变量的重要性', fontsize = 8)
plt.xlabel('输入变量', fontsize = 8)
```

模型评价:

	precision	recall	f1-score	support
drugA	1.00	1.00	1.00	23
drugB	1.00	1.00	1.00	16
drugC	0.00	0.00	0.00	16
drugX	0.77	1.00	0.87	54
drugY	1.00	1.00	1.00	91
accuracy			0.92	200
macro avg	0.75	0.80	0.77	200
weighted avg	0.86	0.92	0.89	200

输入变量重要性:

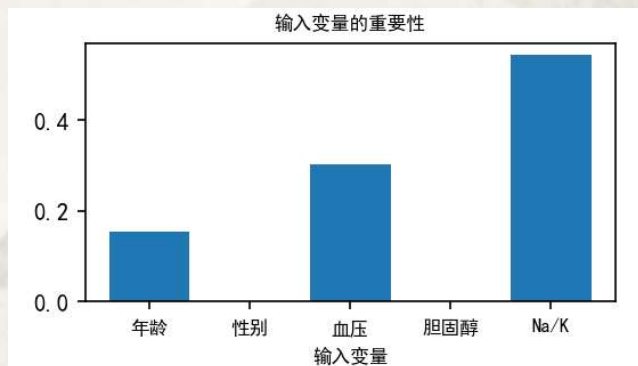
[0.15485334 0. 0.30265555 0. 0.5424911]



- **案例五：药物适用性研究--多分类的预测问题**
 - 目标：探索影响药物适用性的**重要因素**和**用药建议**

```
plt.figure(figsize = (15,10))
tree.plot_tree(modelDTC,feature_names = list(X),class_names = np.unique(y),filled=True)
print(tree.export_text(modelDTC))
```

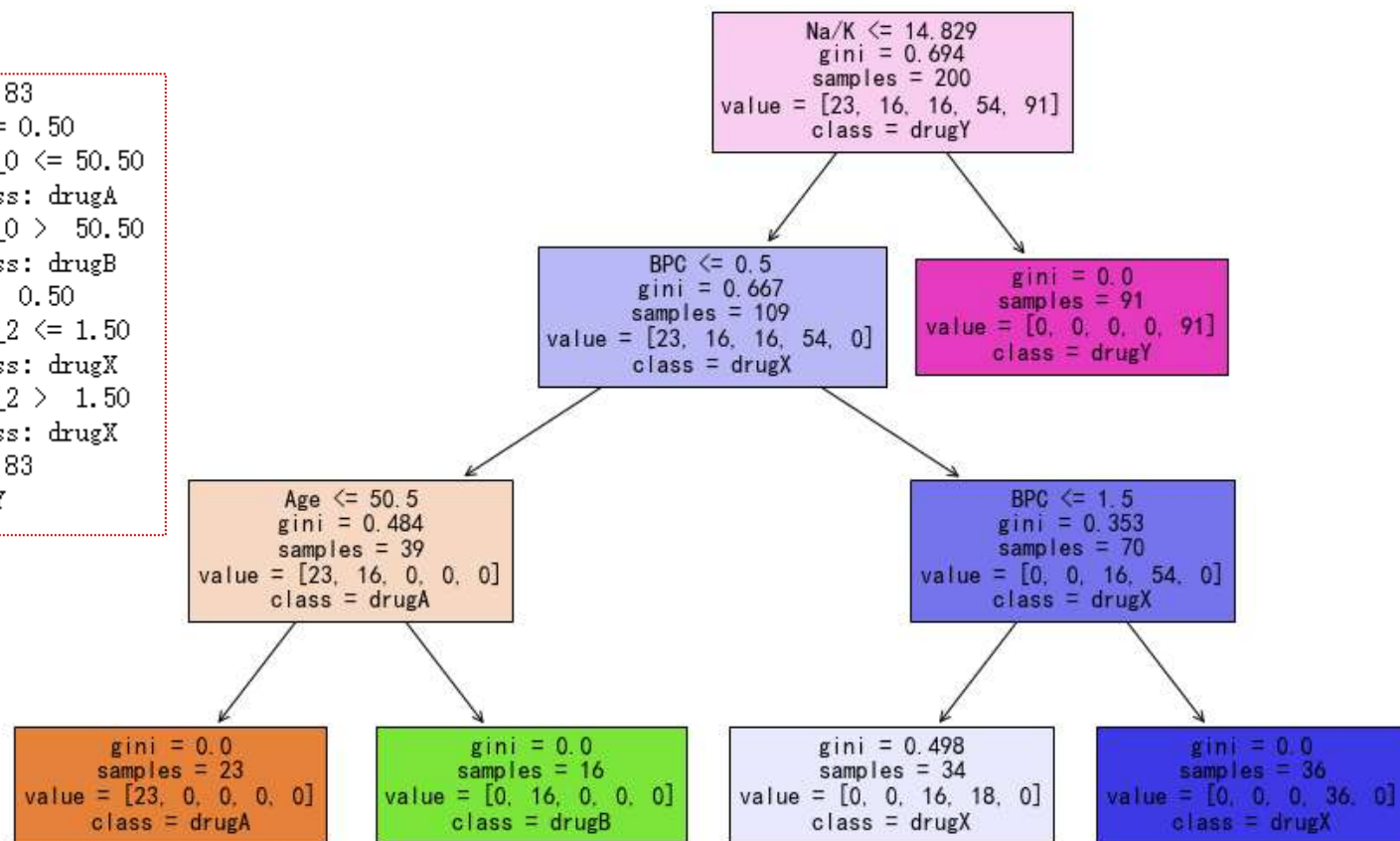
```
X = data[['Age', 'SexC', 'BPC', 'CholesterolC', 'Na/K']]
```



```

|— feature_4 <= 14.83
|   |— feature_2 <= 0.50
|       |— feature_0 <= 50.50
|           |— class: drugA
|       |— feature_0 > 50.50
|           |— class: drugB
|   |— feature_2 > 0.50
|       |— feature_2 <= 1.50
|           |— class: drugX
|       |— feature_2 > 1.50
|           |— class: drugX
|   |— feature_4 > 14.83
|       |— class: drugY

```

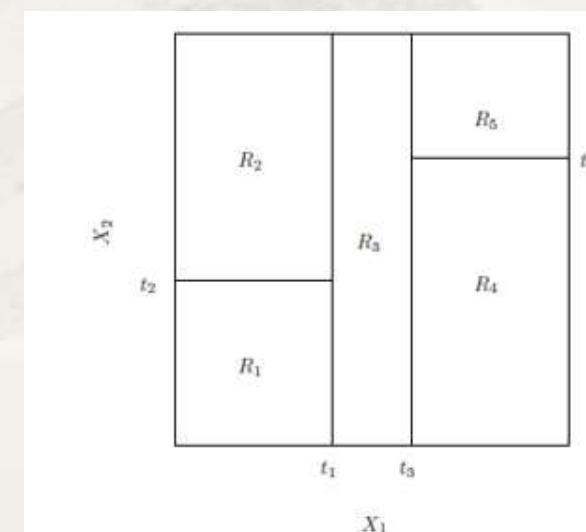
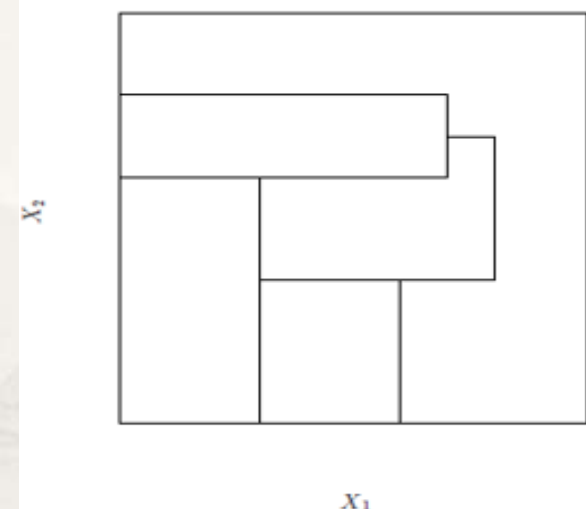


- **问题与思考**
 - 如何解读决策树的输出？
 - 用药建议是什么？

决策树算法的思考

- 优势：模型可解释性强；特别适合分类型输入变量；可度量变量的重要性
- 不足：当前只能考虑一个输入变量
 - 算法聚焦高效实现分组
 - 每次仅进行单变量分组而不是多变量交叉分组
 - 二叉树
- 决策树中的随机性是什么？

```
modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
```



决策树算法的思考

- 再看案例二：变量重要性的再探讨
- 结论：影响两类骑行日租赁量的两大重要因素均为体感温度和湿度
- 问：
 - 有哪些不足？

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
for l, bestDeep, Y in zip([0, 1], [9, 7], ['casual', 'registered']):
    y = data[Y]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
    modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
    modelDTC.fit(X_train, y_train)
    axes[l].bar(np.arange(8), modelDTC.feature_importances_)
    axes[l].set_title('输入变量的重要性(%s)%(Y)', fontsize=14)
    axes[l].set_xlabel('输入变量', fontsize=14)
    axes[l].set_xticks(np.arange(8))
    axes[l].set_xticklabels(X.columns, fontsize=10)
```

